

02. 玩转 VS Code

目录

- 02. 玩转 VS Code
 - 目录
 - 前言
 - 第一章 - Windows、macOS、Linux 安装详解
 - 1.1 Windows 安装
 - 1.2 macOS 安装
 - 1.3 Linux 安装
 - 1.4 code 命令：在终端里打开 VS Code
 - 1.5 这几个配置，让你的 VS Code 更好用
 - 第二章 - 界面布局与核心区域
 - 2.1 活动栏（侧边栏）
 - 2.2 编辑区
 - 2.3 集成终端
 - 2.4 状态栏
 - 2.5 命令面板
 - 第三章 - 写作效率技巧
 - 3.1 快速打开文件：别再在一堆标签页里翻来翻去了
 - 3.2 多光标编辑：同时改多个地方，不用一个一个来
 - 3.3 分屏写作
 - 3.4 保存时自动格式化
 - 3.5 其他实用设置
 - 第四章 - 快捷键：从常用到速查
 - 4.1 Ctrl+P：快速打开文件
 - 4.2 Ctrl+Shift+P：命令面板
 - 4.3 Ctrl+D：选中下一个相同词
 - 4.4 Ctrl+`：打开/关闭终端
 - 4.5 Alt+↑/↓：整行移动
 - 4.6 替换与查找
 - 4.7 快捷键速查表
-

前言

Hello World again!

恭喜你，从《01. 玩转 Markdown》里活着出来了。Kate 也活着出来了——她现在已经能用 Markdown 写出一篇格式清晰的笔记，标题、列表、链接、图片，样样都能来几下。

但学了几天之后，Kate 发现了一个问题。

她写 Markdown 的时候，左手键盘右手鼠标来回切换。敲几个 # 写标题——手在键盘上。然后要打开另一个文件——手挪到鼠标上，去侧边栏点。点完再回来敲键盘。过一会又要格式化表格——又去摸鼠标。

她说：“我总感觉哪里不够帅。”

你的直觉和 Kate 一样。这个直觉是对的。

鼠标是效率的减速带

你在 01 里学会了用 ** 加粗、用 # 写标题——这些都是纯键盘操作，手不离开键盘，思路如流水。

但你的右手还在偷偷摸鼠标：

- 打开另一个文件 → 鼠标去侧边栏点
- 切换文档 → 鼠标去标签栏点
- 格式化表格 → 鼠标去插件菜单点
- 截图粘贴 → 还要先打开截图软件

每一次鼠标移动，都像在高速公路上踩一脚刹车。一次无所谓，但一天踩一百次，你会发现自己的另一半时间不是花在**写上**，而是花在**点上**。

试想一下，如果用 Word 写这些——每一步都要摸鼠标。改个标题字号，你得在工具栏的下拉菜单里翻好几秒；插个图片，你得从菜单栏一层层点进去；想保持代码格式？想都别想，Word 会“贴心”地把你的直引号全弯掉。你在 Word 里花的时间，至少有一半不是在写内容，是在伺候格式。

Kate 算了一下：她写一篇笔记要 30 分钟，其中大概有 10 分钟是在用鼠标找文件、点菜单、拖窗口。如果把这 10 分钟省下来，她每天能多写半篇笔记，或者多喝一杯咖啡。

VS Code 是什么？

Visual Studio Code（简称 VS Code）是微软在 2015 年发布的一款编辑器。

“编辑器”就是用来写字的软件——就像 Word 是用来写文档的，VS Code 是用来写代码和 Markdown 的。但它比 Word 轻得多（打开速度飞快），又比记事本强得多（功能多到数不完）。

它凭什么从几百款编辑器里杀出来，成了全球程序员的首选？

理由	一句话解释
免费 + 开源	不花一分钱。“开源”的意思是它的源代码是公开的，全世界的开发者都可以检查它、改进它。
跨平台	Windows、macOS、Linux 都能装，而且界面一模一样。你在学校用 Windows 学了，回家用 Mac 也能无缝衔接。
插件生态	几万个免费插件，就像一个应用商店。想加什么功能就装什么插件——从 Markdown 到 Python 到 AI 助手。
内置终端	不用切换窗口，编辑器里直接就能敲命令行。Kate 第一次发现这个功能时，说了一句“这也太方便了吧”。
Git 集成	Git 是一个帮你记录文件修改历史的工具。VS Code 把 Git 的功能做进了界面里，你改了什么，旁边一目了然，不用打开命令行慢慢敲。
速度快	启动快，打开大文件也不卡。不像某些软件，启动的时间够你去倒一杯水。
社区庞大	遇到问题，在 Google 或百度上一搜“VS Code 怎么xxx”，全是答案。你基本不会是第一个遇到那个问题的人。

简单说：VS Code 不只是个“写字板”，它是一个**集写作、终端、版本控制、插件扩展于一体的指挥中心**。

在《01. 玩转 Markdown》的第一章，我们帮你把 VS Code 请进了家门，还给它装了六个插件。但说实话——那只是“安装”。

就像你买了一辆法拉利，只学会了怎么发动引擎，还没挂挡上路。你现在会踩油门，但你还不知道这辆车可以漂移、可以弹射起步、可以在任何地形上开。

这套教程，就是你的《**VS Code 驾驶手册**》。Kate 正在驾校门口等你。

 我们会讲

章	内容	学完你能
第一章	Windows、macOS、Linux 安装详解	在任何系统上正确安装 VS Code，配置 <code>code</code> 命令
第二章	界面布局与核心区域	知道活动栏、编辑区、终端、状态栏、命令面板各自干什么，不再对着屏幕一脸懵
第三章	写作效率技巧	快速打开文件、多光标编辑、分屏写作、自动格式化——用键盘速度碾压鼠标流
第四章	快捷键：从常用到速查	详解 5 个最常用的快捷键 + 一张可打印的速查表，练成肌肉记忆

📌 这套教程和 01 的关系

01. 玩转 Markdown	02. 玩转 VS Code
教语言——Markdown 语法	教工具——VS Code 编辑器
学完你能写出好看的文字	学完你能用最快的速度写出好看的文字
剑法	剑

两套教程互相独立，又互相补位。你可以先学 01，再学 02；也可以正在学 01 的时候，遇到某个操作不知道怎么更快，跳到 02 的对应章节翻翻。Kate 是学完 01 再开始 02 的，她说这样顺序正好。

🔧 本教程的规矩

和 01 一样，只有一条：

打开 VS Code，跟着敲。 别开着教程当电视剧看。

每一章都有实战操作。Kate 会在每一章等你——她敲错了，你来分析哪里错了。你敲对了，Kate 会给你发一个 👍。

你没动手，就等于没学。

准备好了吗？

第一章，我们先把 VS Code 在你的系统上**装得明明白白**。不管你是 Windows、Mac 还是 Linux，都有一份专属的安装指南。

守夜人，敲键盘！🔪

第一章 - Windows、macOS、Linux 安装详解

在《01. 玩转 Markdown》里，我们只是匆匆扫了一眼安装步骤——勾两个框，一路 Next。但实际情况是：**不同操作系统，安装方式完全不同，踩的坑也完全不同。**

这一章，我们把三个系统分开讲。你可以只读自己系统的那一节，但建议把三个都扫一遍——以后帮同学装 VS Code 时，你就是他们的“IT 支援部”。

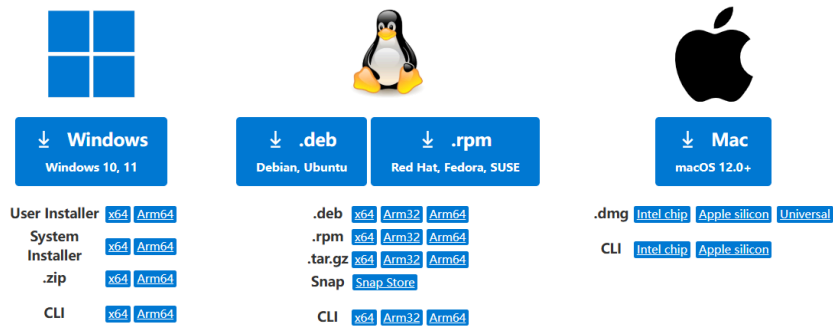
1.1 Windows 安装

第一步：下载

打开 [VS Code 官网](#)，点击那个大大的 **Download for Windows** 按钮。

Download Visual Studio Code

Free and built on open source. AI agents, Git, debugging, and extensions included.



你会下载到一个 `VSCodeSetup-x64-版本号.exe` 文件，大概 80-100MB。

⚠ 别下错了： 有两种安装包——**User Installer** 和 **System Installer**。官网默认给你的是 User Installer，只装给当前用户，不需要“管理员权限”（就是电脑的最高控制权）。如果你想让电脑上所有账号都能用 VS Code，选 System Installer（在下载按钮旁边的箭头里切换）。

Kate 问：“我选哪个？”
“你自己的电脑，选 User Installer 就行。学校机房那种公用电脑才需要 System Installer。”

第二步：安装

双击 `.exe` 文件，进入安装向导。

一路点“下一步”，直到看到“**选择附加任务**”页面。这里是本章的**核心战场**：

选项	勾不勾	原因
添加到右键菜单	☒ 必须勾	以后在任意文件夹上右键 → “Open with Code”，一秒打开。不勾？每次都要先开 VS Code，再 File → Open Folder，多三步
添加到 PATH	☒ 必须勾	勾了才能在终端里输入 <code>code</code> 打开项目。不勾？你会在命令行里看到 <code>'code'</code> 不是内部或外部命令，当场社死
注册为支持的编辑器	随便	不影响使用
创建桌面快捷方式	随便	看你习惯

守夜人铁律： 那两个勾，少一个都不行。这不是“建议”，是“第一章在教你的时候就这么定的”。你在 01教程里已经享受过这两个勾的好处了——右键一秒打开文件夹、终端里 `code` 直接起飞。

其他步骤无脑 Next，安装完成。

第三步：验证

打开终端。什么是终端？在第一章里我们讲过——它是一个黑框框，你可以在里面打字控制电脑。

- 按下键盘上的 `Win + R`（按住 `Win` 键不松手，点一下 `R` 键），会弹出一个叫“运行”的小窗口。
- 在输入框里打 `cmd`，然后回车。
- 一个黑底白字的窗口弹出来了。这就是终端。

在终端里输入：

```
code --version
```

如果跳出来一串版本号（比如 `1.100.0`），恭喜你，装好了。

Kate 看到版本号跳出来的时候，说：“这就装好了？我以为还要填一堆东西。”

1.2 macOS 安装

第一步：下载

打开 [VS Code 官网](#)，点击 **Download for macOS**。

你会下载到一个 `VSCode-darwin-版本号.zip` 文件。

第二步：安装

双击 `.zip` 解压，把解出来的 `Visual Studio Code.app` 拖进“应用程序”文件夹。

什么是“应用程序”文件夹？打开 Finder（Mac 上的文件管理器，图标是一个蓝白笑脸），左边侧栏里有一个叫“应用程序”的文件夹。把 `Visual Studio Code.app` 拖进去就行。

⚠️ 新手常见错误：很多 Mac 用户直接在下载文件夹里双击 `.app`，用了一周后发现“怎么每次都要重新下载？”——因为它根本没被安装，只是临时运行。

守夜人铁律：拖进“应用程序”文件夹，才算真正“装好”。

第三步：配置 `code` 命令 (macOS 重点)

在 Windows 上，安装向导帮你勾了 `PATH`。macOS 没有这一步，需要手动操作：

1. 打开 VS Code
2. 按 `Cmd+Shift+P`。`Cmd` 键在空格键旁边，上面画着一个 `⌘` 符号。按住它不松手，再按 `Shift` 和 `P`。这会打开一个叫“命令面板”的东西——屏幕顶部中间会弹出一个输入框。
3. 在输入框里打 `shell command`。你会看到一个选项叫 **“Shell Command: Install 'code' command in PATH”**。

4. 点它，回车。

完成后，打开终端（在“应用程序” → “实用工具” → “终端”，或者按 `Cmd+空格` 输入“终端”），输入：

```
code --version
```

跳出版本号就是成功了。以后在任何文件夹里，输入 `code .` 就能用 VS Code 打开当前目录。

1.3 Linux 安装

Linux 的安装方式取决于你的“发行版”。什么是发行版？Linux 不是一个单一的系统——它有很多“版本”，叫发行版。最常见的有 Ubuntu、Fedora、Arch Linux。不同发行版的安装命令不一样。

以下是最常见的三种：

Ubuntu / Debian 系列

```
sudo apt update
sudo apt install wget gpg
wget -qO- https://packages.microsoft.com/keys/microsoft.asc | gpg --dearmor > packages.microsoft.gpg
sudo install -D -o root -g root -m 644 packages.microsoft.gpg /etc/apt/keyrings/packages.microsoft.gpg
sudo sh -c 'echo "deb [arch=amd64,arm64,armhf signed-by=/etc/apt/keyrings/packages.microsoft.gpg] https://packages.microsoft.com/repos/visual-studio-code main amd64 arm64 armhf" > /etc/apt/sources.list.d/visual-studio-code.list'
rm -f packages.microsoft.gpg
sudo apt update
sudo apt install code
```

简单说： 添加微软的软件源，然后 `apt install code`。一行 `code` 命令就装好了。

Fedora / RHEL 系列

```
sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc
sudo sh -c 'echo -e "[code]\nname=Visual Studio Code\nbaseurl=https://packages.microsoft.com/yum-repo/visual-studio-code\nenabled=1\ngpgcheck=1\ninstallonly=1" > /etc/yum.repos.d/microsoft.repo'
sudo dnf install code
```

Arch Linux

```
sudo pacman -S code
```

Arch 用户：就这一行。你们懂的。

Snap (通用方式，但不太推荐)

```
sudo snap install code --classic
```

Snap 版能用，但有两个问题：启动比正常版本慢好几秒，插件更新也经常滞后。能不用就不用。

守夜人建议： 不管你用哪个发行版，优先用官方软件源安装，别去“软件中心”点——那里的版本经常是社区打包的，缺功能。

1.4 code 命令：在终端里打开 VS Code

Windows 用户安装时勾了 PATH，macOS 用户手动配置了 `shell command`，Linux 用户用包管理器安装时自动加了——现在三个系统都有了 `code` 命令。

这是你 VS Code 生涯的**第一个效率工具**，也是最常用的一个。

1.4.1 基础用法

命令	作用	什么时候用
<code>code .</code>	用 VS Code 打开 当前目录	打开一个项目文件夹
<code>code 文件名.md</code>	用 VS Code 打开 单个文件	快速编辑某个文档
<code>code 文件夹名</code>	用 VS Code 打开 指定文件夹	打开不是当前目录的项目

怎么用？

打开你的终端：

- Windows： `Win+R` → 输入 `cmd` → 回车
- macOS： `Cmd+空格` → 输入 `终端` → 回车
- Linux： `Ctrl+Alt+T`

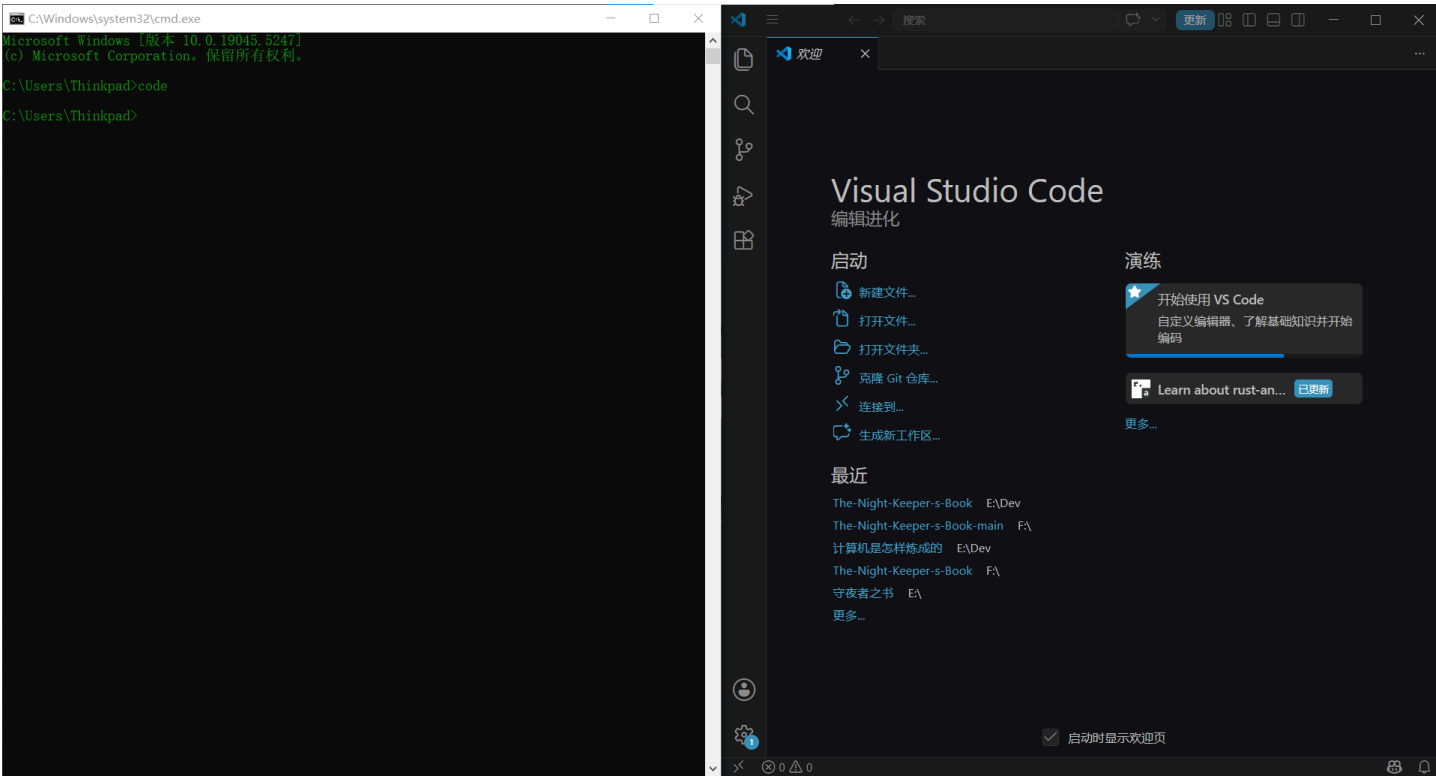
然后直接敲 `code`，回车。VS Code 就打开了。

如果你想打开一个文件夹，不用先启动 VS Code 再 `File` → `Open Folder`。在终端里 `cd` 到那个文件夹，输入：

```
code .
```

`.` 代表“当前目录”。VS Code 会瞬间启动，并且左侧文件列表已经显示了这个文件夹里的所有文件。

以后你再也不需要“先打开 VS Code → File → Open Folder → 找到文件夹 → 点确定”。四步操作，`code .` 一步搞定。



Kate 第一次用 `code .` 的时候，愣了一下，然后说：“这就打开了？我连鼠标都没碰。”

1.4.2 窗口管理：重用 vs 新建

默认情况下，每次执行 `code .` 都会在同一窗口里打开。如果你想让新项目在一个新窗口里打开：

```
# 在新窗口中打开（不关掉原来的窗口）
code -n .

# 在已有窗口中打开（替换当前项目）
code -r 新文件夹名
```

参数	全称	作用
-n	--new-window	强制打开一个新窗口
-r	--reuse-window	在已打开的窗口中打开文件或文件夹

守夜人心法：同时写两个项目时用 `-n`，各开各的窗口。只写一个项目时用 `-r`，或者什么都不加（默认就是重用窗口）。

Kate 现在左边屏幕开着 01 的 Markdown 教程，右边屏幕开着 02 的 VS Code 教程。她说：“`code -n` 就是我的分身术。”

1.4.3 跳转到指定行

代码报错了，报错信息告诉你 `main.py:42` ——意思是 `main.py` 的第 42 行出了问题。

什么是“行号”？打开 VS Code，编辑区左边有一排数字——1、2、3、4……这些就是行号。每一行代码都有一个编号，方便你快速定位。

你可以这样直接跳到那一行：

```
code -g main.py:42
```

参数	全称	作用
-g	--goto	打开文件并 跳转到指定行

后面还可以加列号（第几个字符）：

```
code -g main.py:42:5
```

跳到 `main.py` 第 42 行第 5 个字符。

1.4.4 对比两个文件

想知道 `旧版本.md` 和 `新版本.md` 有什么区别？不用装任何插件，VS Code 自带对比功能：

```
code -d 旧版本.md 新版本.md
```

参数	全称	作用
-d	--diff	打开 两个文件并排对比 差异

效果：左右两个编辑区，改动的行被高亮标出来。写教程改稿子时特别有用——旧版和新版一字排开，哪里改了哪里没改，一目了然。

Kate 用这个功能对比了她昨天和今天的 Markdown 笔记。她指着左边被划掉的一行字说：“这是我昨天写的‘Python 好难’。右边没了。”“你现在怎么想？”“……Python 还是好难。但至少我不用再写两遍笔记了。”

1.4.5 命令行安装插件

不用打开 VS Code 的扩展商店，直接在终端里装插件：

```
code --install-extension 插件ID
```

比如装 Markdown 相关插件：

```
code --install-extension yzhang.markdown-all-in-one
code --install-extension shd101wyy.markdown-preview-enhanced
```

这在重装系统后特别有用：把所有插件 ID 写成一个脚本，一行命令全部装回来。

1.4.6 查看所有参数

如果你想知道 `code` 命令还支持什么参数：

```
code -h
```

或者

```
code --help
```

会列出所有可用的命令行参数。

1.4.7 速查表

命令	作用
<code>code .</code>	打开当前目录
<code>code 文件名</code>	打开指定文件
<code>code -n .</code>	在新窗口打开当前目录
<code>code -r 文件夹</code>	在已打开窗口里替换项目
<code>code -g 文件:行号</code>	打开文件并跳转到指定行
<code>code -d 文件1 文件2</code>	并排对比两个文件
<code>code --install-extension ID</code>	在终端里装插件

命令	作用
<code>code -h</code>	查看所有参数

这就是为什么我们在第一章就强调**那两个勾必须打上**。不打勾，`code` 命令根本不能用——你就永远体验不到“一个命令打开一个世界，再一个参数跳到你想要的那一行”的快乐。

1.5 这几个配置，让你的 VS Code 更好用

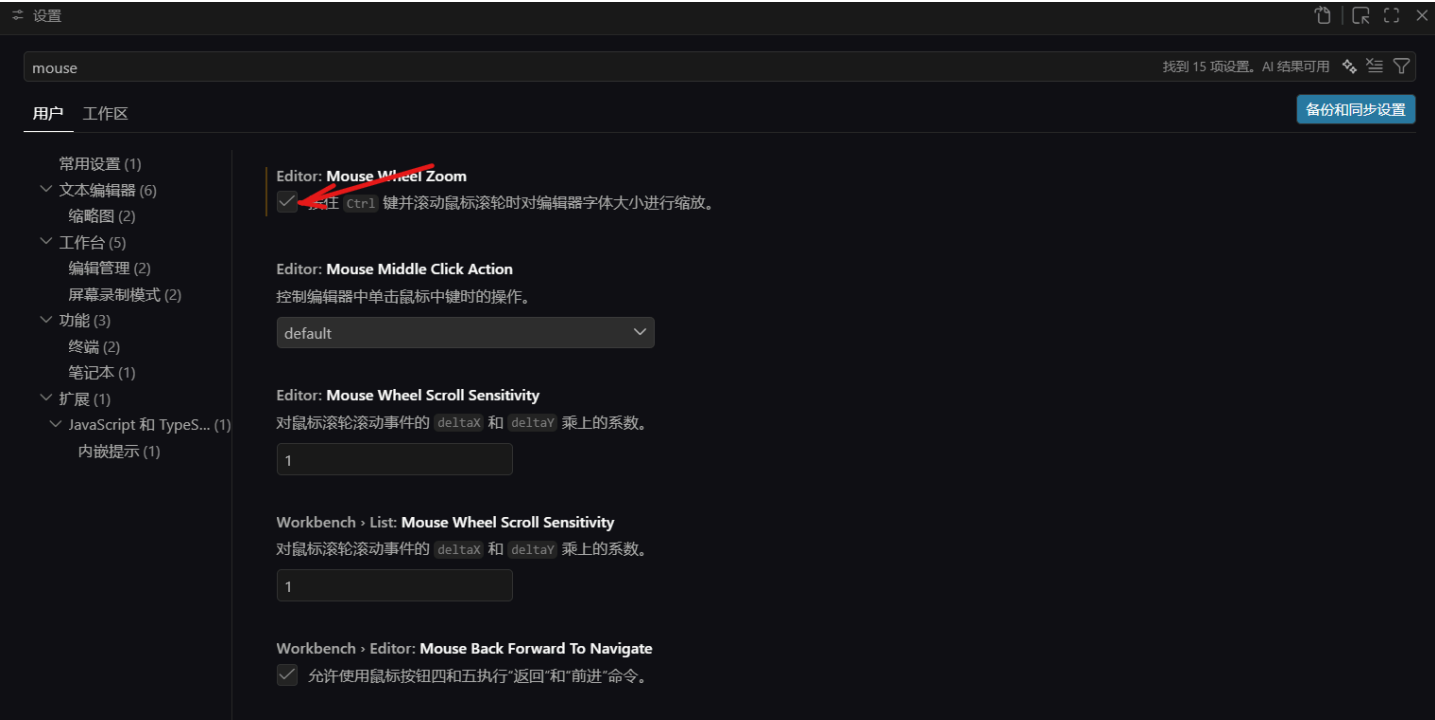
1.5.1 缩放字体

有时候你在给别人演示代码，希望字体大一点，让后排的人也能看到。有时候你在认真写文章，希望字体小一点，屏幕上能多显示几行。

VS Code 支持用 `Ctrl + 滚轮` 来缩放字体，但这个功能**默认是关闭的**。你需要先打开它。

第一步：开启鼠标滚轮缩放

1. 按 `文件 → 首选项 → 设置`（或按快捷键 `Ctrl + ,`，Mac 上是 `Cmd + ,`）打开设置页面。
2. 在搜索框里输入 `mouse`。
3. 找到一个叫 **“Editor: Mouse Wheel Zoom”** 的选项。它默认是**没勾选**的。
4. **勾上它。**



第二步：使用

勾上之后，你就可以按住 `Ctrl`（Mac 上是 `Cmd`），然后滚动鼠标滚轮。往前滚放大字体，往后滚缩小字体。松开 `Ctrl`，字体大小就固定了。

这个操作只影响当前窗口的显示大小，不会改变文件本身的内容。就像你用两个手指在手机上缩放照片一样——照片没变，只是你看到的大小变了。

Kate 勾上这个选项，把字体放大到 150%，然后往后一靠，说：“这下可以躺着写笔记了。不过这功能竟然默认是关的——藏的也太深了。”

1.5.2 换主题：给你的编辑器换件衣服

VS Code 默认的深色主题叫“Dark+”。如果你不喜欢深色，或者看久了觉得累，可以换一个。

1. 按 `Ctrl+K`，松开，再按 `Ctrl+T`（Mac 上是 `Cmd+K` 然后 `Cmd+T`）。注意这不是同时按——先按 `Ctrl+K`，松开，再按 `Ctrl+T`。
2. 屏幕中间会弹出一个主题列表。用上下箭头键预览不同的主题——你每移到一个主题上，VS Code 会立刻显示那个主题的效果，不用点确认。
3. 选一个你喜欢的，回车确定。

主题名	风格	适合
Dark+	深色，VS Code 默认	长时间写作，眼睛不容易累
Light+	浅色，白底黑字	白天光线强的时候看得更清楚
Monokai	深色，咖啡色调	很多程序员的最爱，看起来像老式终端
Solarized Light	浅色，暖黄色调	护眼，适合读代码多过写代码的人

守夜人建议：如果你每天写 Markdown 超过两个小时，用深色主题。浅色主题在长时间看屏幕时会比深色更刺眼。当然，这完全看个人喜好——Kate 用的是 Dark+，她说“深色让我觉得自己是黑客”。

Kate 把主题列表从头到尾翻了一遍，最后选了 Dark+。她说：“还是默认的好看。但至少我知道怎么换了。”

1.5.3 文件自动保存

你写了半天笔记，电脑突然蓝屏了——重启之后发现，之前写的内容全没了。因为你忘了保存。

VS Code 可以帮你自动保存，不用每次按 `Ctrl+S`。

1. 按 `Ctrl+,`（Mac 上是 `Cmd+,`）打开设置页面。

- 2. 在搜索框里输入 `auto save`。
- 3. 找到一个叫 **“Files: Auto Save”** 的选项。它默认是 `off`（关闭）。
- 4. 把它改成 `afterDelay`（延迟后自动保存）。下面会出现一个“Auto Save Delay”的选项，默认是 1000 毫秒，也就是 1 秒。你停止打字 1 秒之后，VS Code 自动帮你保存。

不用担心“万一我写错了它自动保存了怎么办”——你随时可以 `Ctrl+Z` 撤销，自动保存不影响撤销。

Kate 打开自动保存之后，说：“我再也不用每写一行就按一次 `Ctrl+S` 了。我的左手小拇指终于可以休息了。”

1.5.4 调整字体大小（固定值）

`Ctrl+滚轮` 是临时缩放。如果你想永久改变字体大小，可以在设置里调。

- 1. 按 `Ctrl+,`，打开设置。
- 2. 搜索 `font size`。
- 3. 找到 **“Editor: Font Size”**，默认是 14。你可以改成 16 或 18，看你自己舒服。

这个设置会影响编辑区的字体大小。如果你想调整终端（VS Code 里面那个黑框框）的字体大小，搜索 `terminal font size`，单独调。

1.5.5 总结

你想干嘛	怎么操作
临时放大/缩小字体	<code>Ctrl + 滚轮</code>
换主题	<code>Ctrl+K</code> 然后 <code>Ctrl+T</code>
自动保存	设置里搜 <code>auto save</code> ，改成 <code>afterDelay</code>
永久改字体大小	设置里搜 <code>font size</code> ，改数字

守夜人心法： VS Code 的设置多到可以出一本书。你不需要全部搞懂。把上面这四个调好，你就能舒舒服服地写 Markdown 了。其他的设置，等你遇到“这个能不能改”的冲动时再搜——大概率能改，而且只用改一个选项。

Kate 调完这四个设置之后，看着她的 VS Code，说：“现在它是我的了。不是默认的那个 VS Code，是我的 VS Code。”

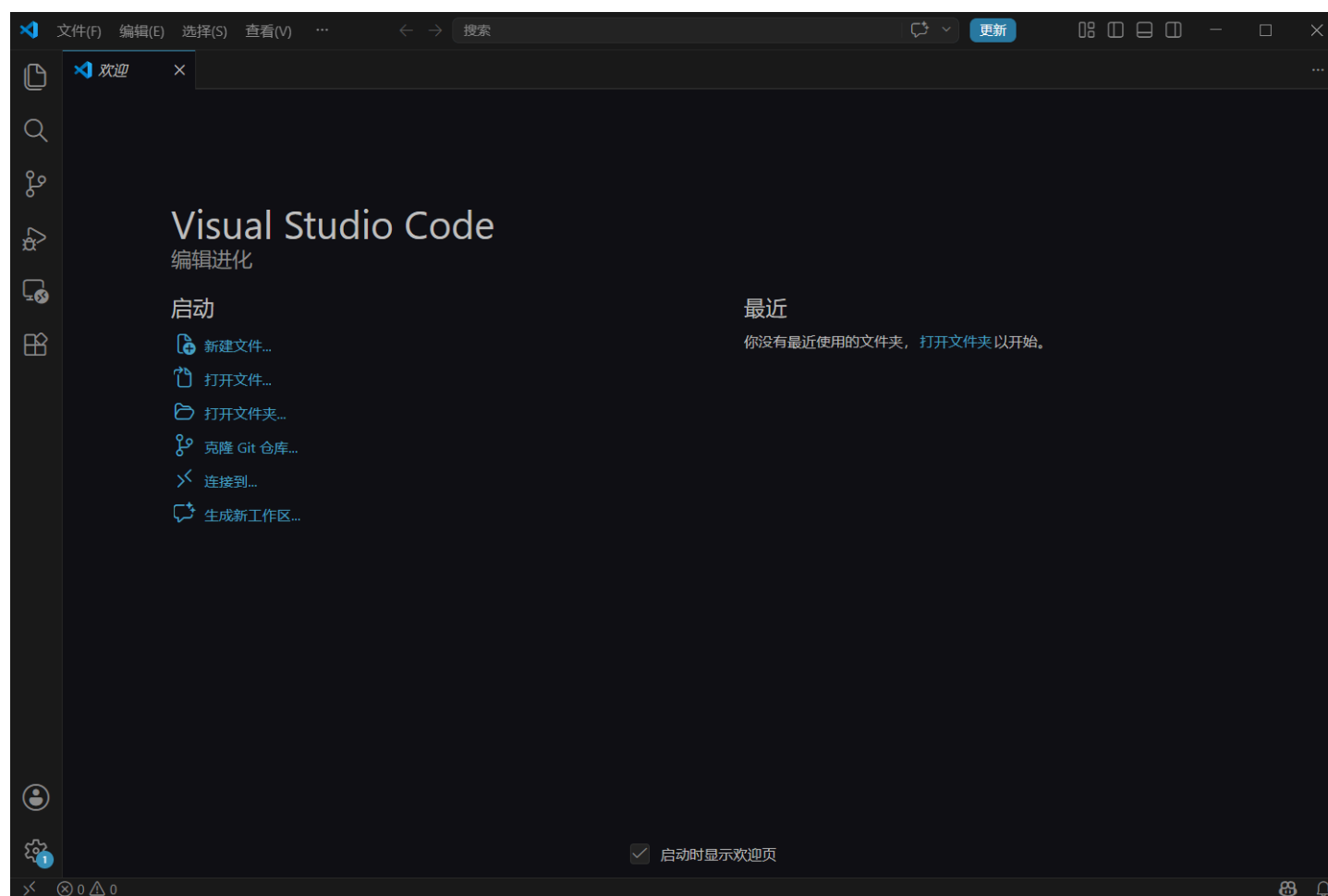
VS Code 已经在你电脑上安家落户了，而且它现在不是“默认设置”，是**你的设置**。

下一章，我们逛逛它的“房间”——活动栏、编辑区、终端、状态栏、命令面板，每个区域干什么，我们一个个认。

守夜人，继续敲！🔪

第二章 - 界面布局与核心区域

在第一章，我们把 VS Code 装好了，也配置了 `code` 命令。现在，打开 VS Code，你会看到这样一个界面。



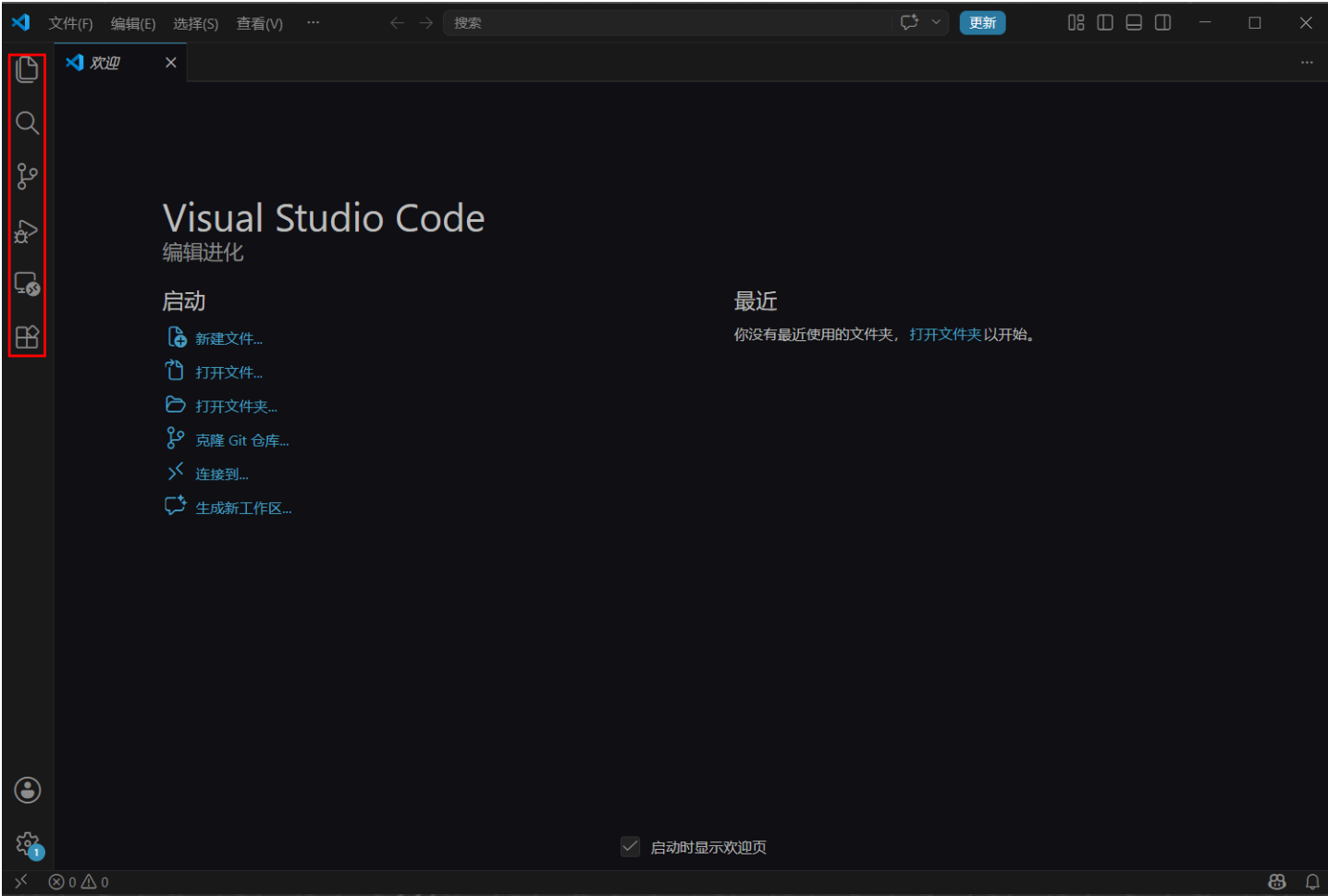
这一章，我们要把这张图里的每一个区域逐个认一遍。学完之后，你不需要记住所有东西，但当你听到“活动栏”、“命令面板”、“终端”这些词时，脑子里能立刻浮现出它们的位置和用途。

Kate 说：“我用了 VS Code 好几天，一直只知道中间那个打字的地方。左边那一排图标我从来没碰过——我怕点错了什么，然后 VS Code 炸了。”

“你不会把 VS Code 点炸。最坏的情况就是关掉重开。今天就放心点。”

2.1 活动栏（侧边栏）

活动栏是 VS Code 左边那一排竖着的图标。它让你在不同功能之间切换——就像一个抽屉柜，每个图标是一个抽屉，点一下就拉开。



从上到下，依次是：

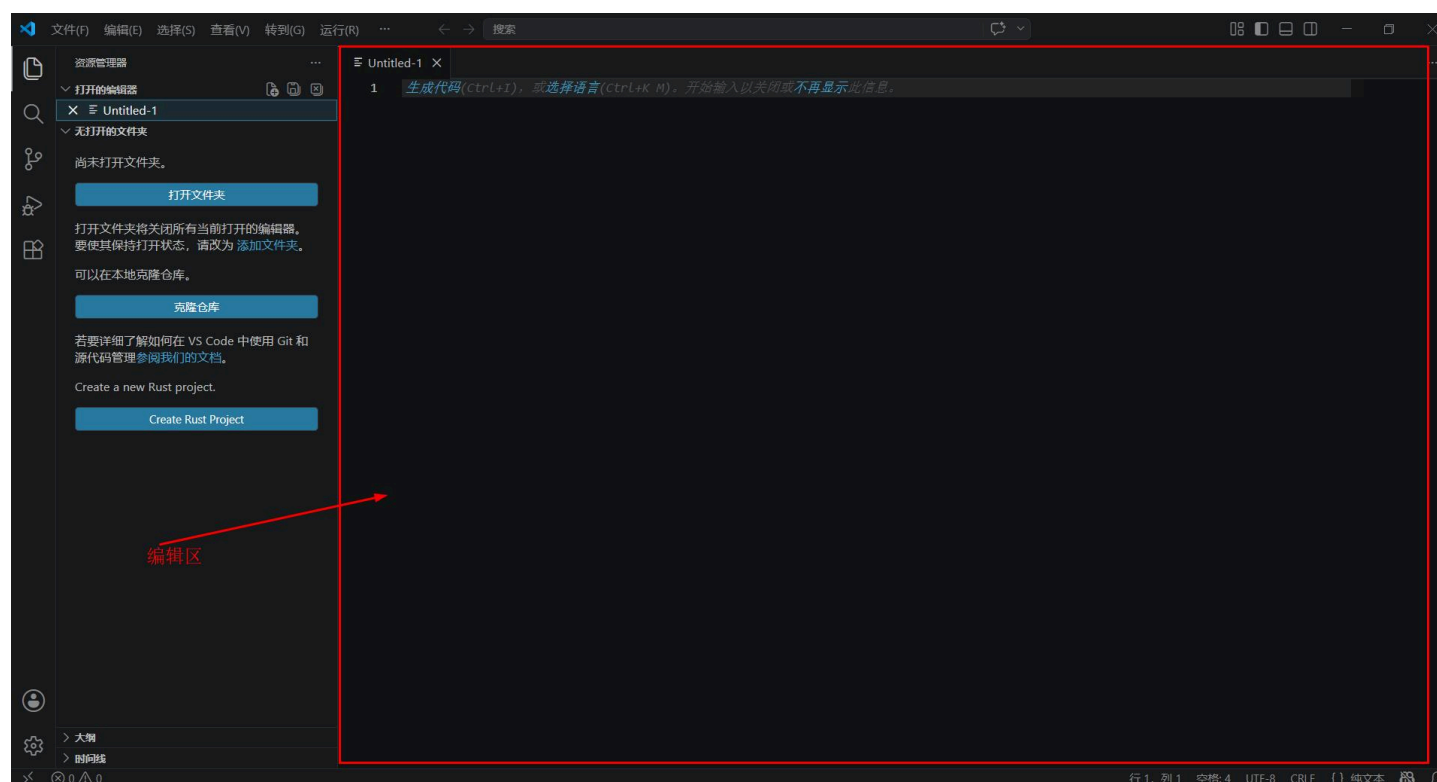
图标	名称	功能
	资源管理器	看文件列表。打开文件夹、新建文件、重命名、删除——都在这里
	搜索	在文件夹里搜内容。可以搜文件名，也可以搜文件里的文字
	源代码管理	Git 的界面。如果你用 Git 管理代码，这里能看到改了哪些文件、谁改的
	运行和调试	跑代码、调 bug。你以后写 Python 脚本时会用到
	扩展商店	装插件的地方。你在 01 教程里装的六神装就是在这里搜的

资源管理器是你最常用的一个——打开文件夹、新建 .md 文件、在文件之间切换，都在这里。

Kate 点了一下资源管理器图标，左边弹出一个文件列表。她说：“原来我的文件都在这里。我以前每次打开文件都是 File → Open，一层层翻文件夹——翻到地老天荒。”

2.2 编辑区

编辑区是 VS Code 中间最大的那一块。你敲的每一个字、写的每一行 Markdown、看的每一段代码，都在这里。



编辑区有几个常用操作：

- **打开文件：** `Ctrl+O`，或者直接在资源管理器里点文件名
- **新建文件：** `Ctrl+N`
- **保存文件：** `Ctrl+S`
- **关闭当前标签页：** `Ctrl+W`

打开的每个文件都显示在**标签栏**上（编辑区顶部那一排文件名）。你可以同时打开多个文件，用 `Ctrl+Tab` 在它们之间快速切换。

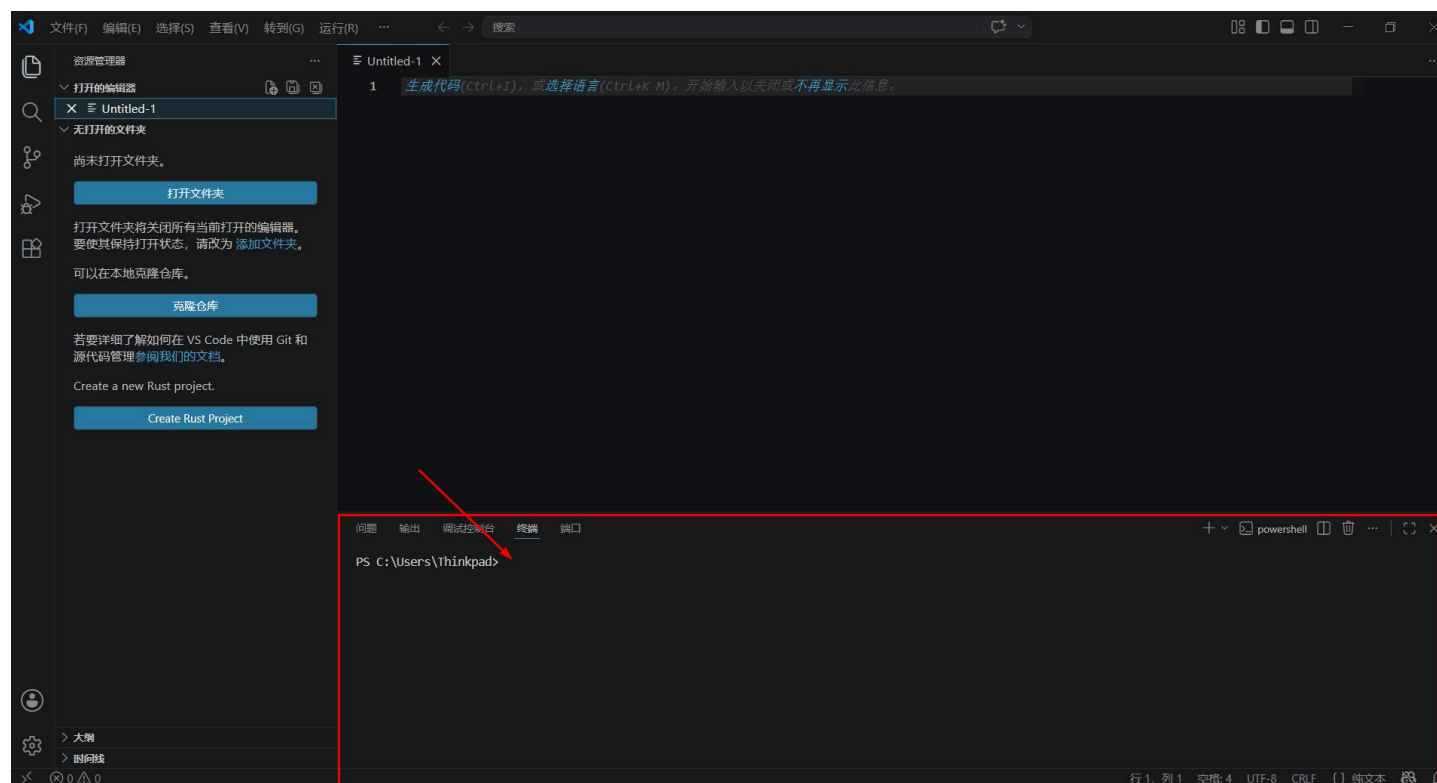
分屏编辑：按 `Ctrl+\`（Mac 上是 `Cmd+\`）把编辑区分成两半。左边写 Markdown 源码，右边看预览；或者左边看参考文章，右边自己写总结。

Kate 按了一下 `Ctrl+\`，编辑区变成了两半。她说：“所以我可以左边写笔记、右边看参考文章？不用 `Alt+Tab` 来回切？”

“对。分屏是‘一边读一边写’的最佳姿势。”

2.3 集成终端

集成终端是 VS Code 底部的一个面板——它就是一个嵌入在编辑器里的终端。你不用切换窗口，在编辑器里就能敲命令。



打开和关闭：

- **打开：**按 `Ctrl+``（键盘左上角 `ESC` 下面的那个反引号键）
- **关闭：**再按一次 `Ctrl+``，或者点终端面板右上角的 `x`

你在这个终端里可以做的事：

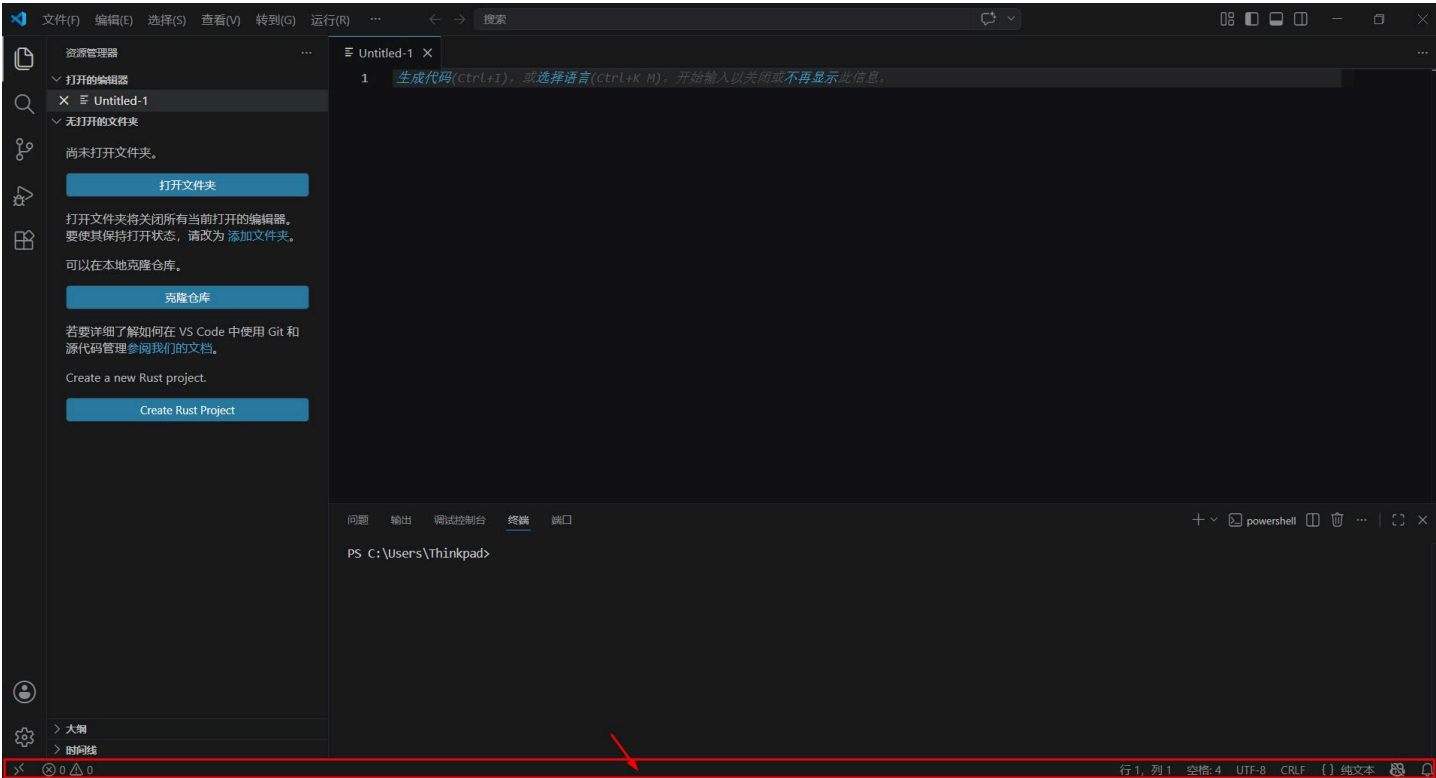
- 用 `code .` 打开另一个项目
- 用 `git status` 看修改了哪些文件
- 用 `pip install` 装 Python 包
- 用 `mkdir` 新建文件夹、`ls` 查看文件列表

这些命令不需要完全理解。你只需要知道：**这个终端和系统终端是一样的——只是它刚好嵌在了 VS Code 里面，不用你来回切窗口。**

Kate 按了 `Ctrl+``，底部弹出一个终端面板。她输入 `ls`，看到了当前文件夹里的所有文件。她说：“原来我不用最小化 VS Code 去开 PowerShell——它自己就有。这就是你 02 前言里说的‘指挥中心’。”

2.4 状态栏

状态栏是 VS Code 最底部那一行字。你平时可能不会注意它，但它能告诉你几个关键信息：



区域	显示什么	有什么用
左侧	Git 分支名、最近修改	知道自己在哪个分支上工作
中间	错误和警告数量	点一下能看到所有问题
右侧	文件编码（UTF-8）、行尾序列（LF/CRLF）、语言模式	点一下可以切换

你可能会用到的操作：

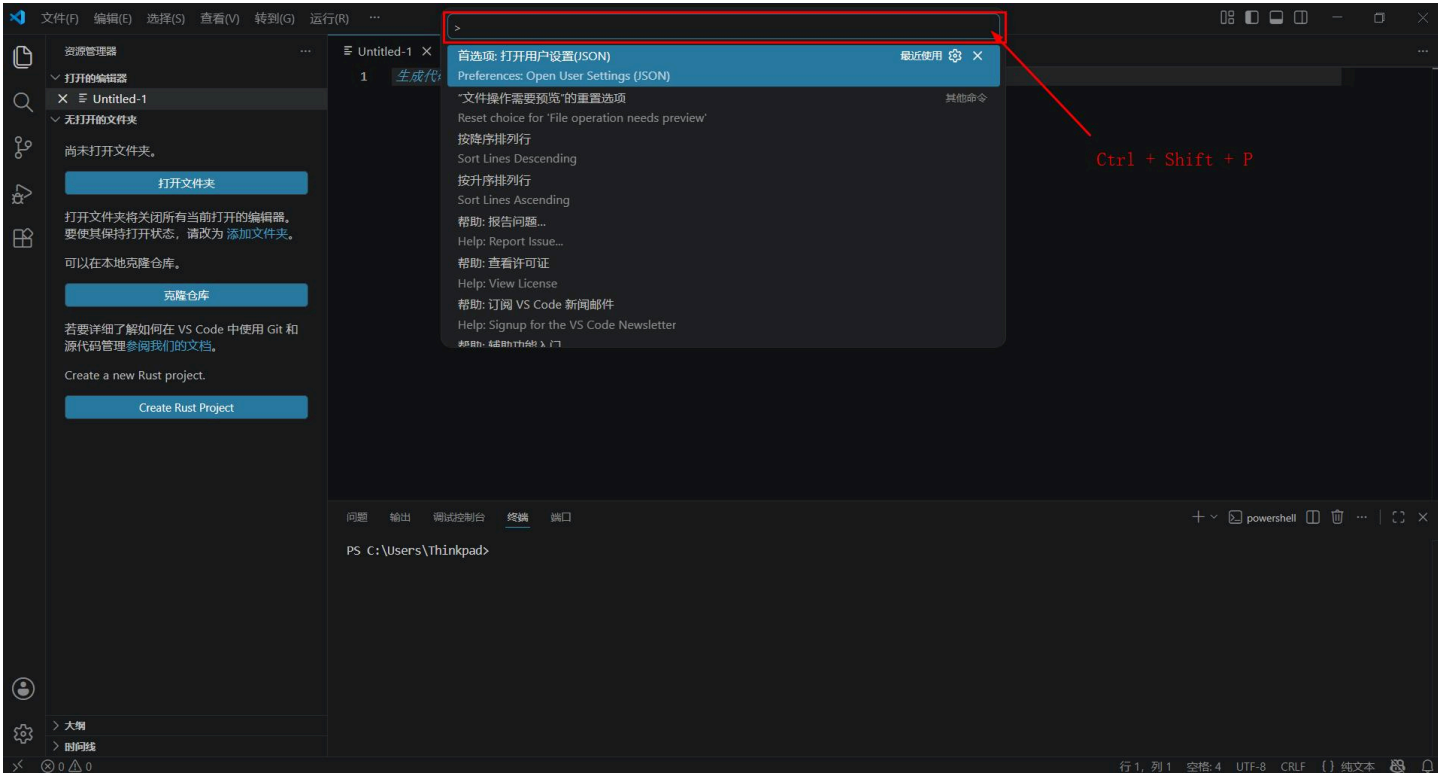
- 点一下右下角的语言模式（比如 `Markdown`），可以切换文件的语法高亮模式
- 如果 Git 显示你有 3 个修改未提交，点一下就能看到改了哪些文件

Kate 看了一眼状态栏，说：“原来下面那行字不是装饰。它告诉我当前文件是什么格式、有没有错误、Git 状态——”

“对。状态栏是 VS Code 的‘仪表盘’。它不主动说话，但你需要信息的时候，低头看一眼就能找到。”

2.5 命令面板

命令面板是 VS Code 的“万能入口”。你不需要记住所有快捷键——只需要记住 `Ctrl+Shift+P`，然后输入你想做的事。



按 `Ctrl+Shift+P`（Mac 上是 `Cmd+Shift+P`），屏幕顶部中间会弹出一个输入框。这就是命令面板。

你可以在命令面板里做几乎任何事：

输入	结果
Create Table of Contents	自动生成 Markdown 目录
Format All Tables	一键对齐所有表格
Export to PDF	导出 PDF
Preferences: Open Settings	打开设置
View: Toggle Terminal	打开/关闭终端

命令面板的好处是：**你不需要记住快捷键。你只需要知道“我想做什么”，然后用文字描述它。** 命令面板是模糊匹配的——你输入 `table`，它会列出所有和表格相关的命令。

Kate 输入了 `format`，命令面板弹出了一堆和格式化相关的选项。她说：“所以我不需要记住 `Ctrl+Shift+I` 是格式化代码——我只需要记住 `Ctrl+Shift+P` 然后输入 `format`。”

“精准。命令面板是快捷键的替代品。你只需要记住一个快捷键——`Ctrl+Shift+P`——然后告诉它你想做什么。”

🎉 第二章，通关！

你现在已经可以：

- ✅ 用活动栏在文件列表、搜索、Git、插件商店之间切换
- ✅ 在编辑区打开、编辑、保存、分屏
- ✅ 用集成终端在编辑器里敲命令，不用切换窗口
- ✅ 从状态栏获取文件格式、Git 状态、错误信息
- ✅ 用命令面板做任何事，不需要记住所有快捷键

Kate 在 VS Code 里从左到右点了一遍：活动栏、编辑区、终端、状态栏、命令面板。点完之后她说：“原来我的 VS Code 不是‘一个打字的窗口’，是五个区域组成的指挥中心。我以前只用了一个——编辑区。剩下四个全没用过。”

下一章，我们要学**写作效率技巧**——快速打开文件、多光标编辑、分屏写作、自动格式化……让你用键盘速度碾压鼠标流。

守夜人，继续敲！🔪

第三章 - 写作效率技巧

3.1 快速打开文件：别再在一堆标签页里翻来翻去了

你在 VS Code 里是怎么打开文件的？

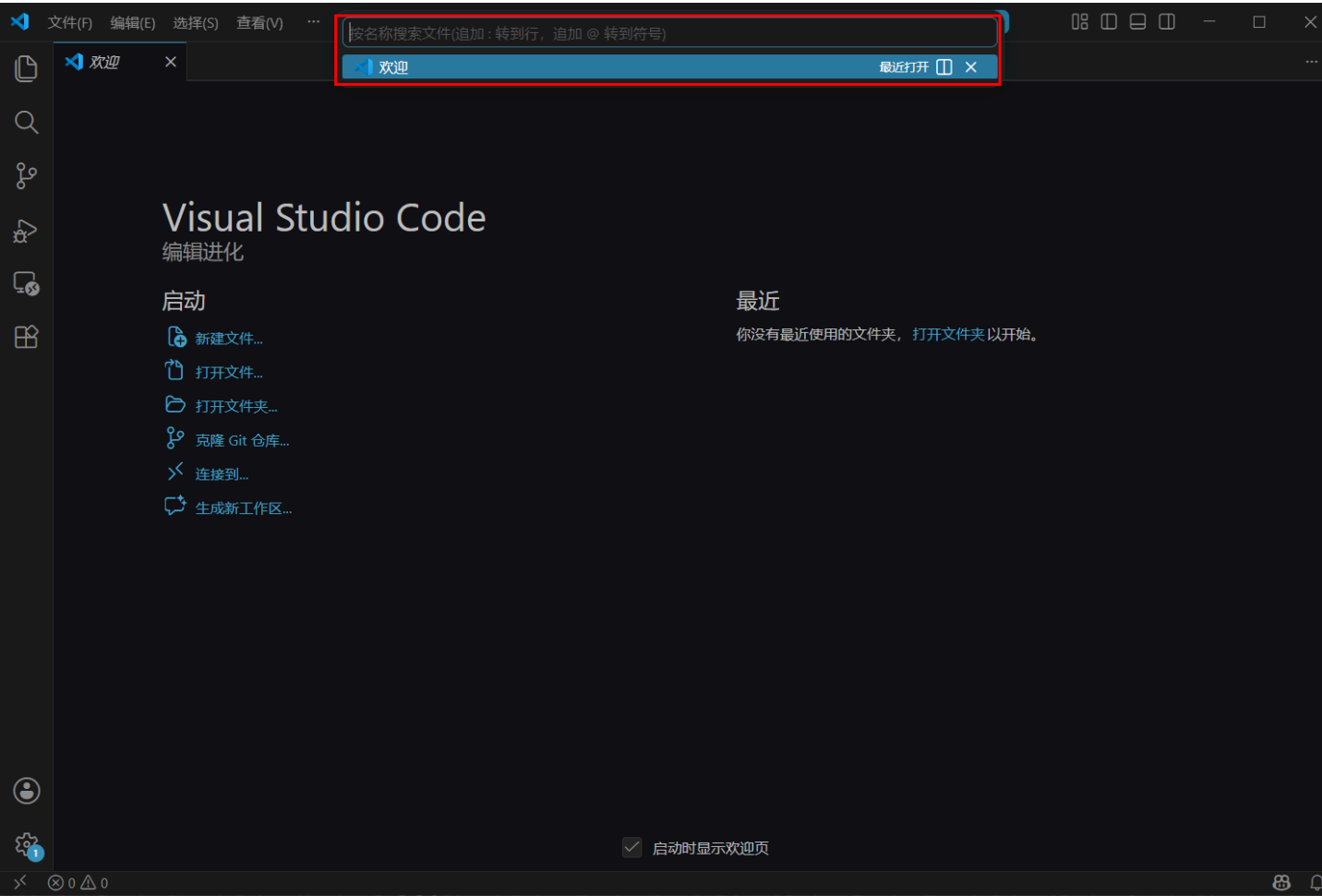
是不是这样：左手离开键盘 → 摸到鼠标 → 眼睛看向左侧的“资源管理器” → 在一堆文件夹里一层一层地找 → 点一下文件名 → 手回到键盘上。

如果你同时开着七八个文件，这个动作你一天可能要重复几十次。每一次都在打断你的写作思路，让你的手离开键盘去摸鼠标，再回来重新找到刚才敲到一半的那一行。

快速打开文件，就是要消灭这个“切换成本”。

3.1.1 快捷键： `Ctrl+P`

按 `Ctrl+P`（Mac 上是 `Cmd+P`），屏幕顶部中央会弹出一个搜索框。



在这个搜索框里，输入文件名的一部分——不需要完整的文件名，不需要完整的路径。只需要几个字母，VS Code 会模糊匹配所有你当前打开过的文件。

比如，你想打开 `01-玩转-Markdown.md`，你只需要输入 `01` 或 `mark` 或 `md` 中的任意几个字母，文件名就会出现在列表里。按回车，文件立刻打开。

Kate 按了 `Ctrl+P`，输入了 `mark`，她的 Markdown 教程文件瞬间出现在列表最前面。她按了回车，文件打开了。她说：“我以前每次打开这个文件，都是去侧边栏翻——先找到文件夹，再找到子文件夹，再在一堆文件里找。现在只需要 `Ctrl+P` 然后打几个字母。这中间我省了多少秒？”

“大概每次 3-5 秒。一天几十次，就是几分钟。一年下来，可能是几十个小时——几十个小时你不去找文件，而是在写内容。”

3.1.2 搜索技巧

`Ctrl+P` 不只是“输入文件名”，它有几个非常实用的搜索技巧：

技巧	输入方式	作用	示例
模糊匹配	文件名里的任意几个字母	快速缩小列表范围	输入 <code>pla</code> 能匹配到 <code>01-玩转-Markdown.md</code>

技巧	输入方式	作用	示例
路径匹配	文件夹名 + 文件名	在特定文件夹里找文件	输入 <code>chapter/01</code> 只匹配 <code>chapter</code> 文件夹里的 <code>01</code> 文件
最近打开	直接按 <code>Ctrl+P</code> , 不看列表	列表默认按最近打开排序	你刚关掉的文件排在最上面, 直接回车就打开

守夜人心法： `Ctrl+P` 是你一天里用得最多的快捷键。把它刻进肌肉记忆——每当你需要打开另一个文件，第一反应不是去摸鼠标，而是按下 `Ctrl+P` 。

3.1.3 和侧边栏的关系

`Ctrl+P` 不是要取代侧边栏——侧边栏适合**浏览**你不知道名字的文件，而 `Ctrl+P` 适合**快速定位**你已经知道名字的文件。两者是互补的，不是互斥的。

- 知道文件叫什么 → `Ctrl+P` , 几个字母定位
- 不知道文件叫什么，想浏览整个项目结构 → 侧边栏，一层层展开
- 刚关掉的文件 → `Ctrl+P` , 它排在最上面

Kate 说：“所以我不用在侧边栏和 `Ctrl+P` 之间二选一——知道名字用 `Ctrl+P`，不知道名字用侧边栏。”

3.2 多光标编辑：同时改多个地方，不用一个一个来

你有没有遇到过这种情况：一篇笔记里，你写错了同一个词——比如把“守夜者”写成了“守夜人”——这个词出现了五六次。你一个一个个地找到、选中、修改、再找下一个。改了五次，手酸眼累。

多光标编辑，就是要让你同时改完所有这些地方，一次搞定。

3.2.1 快捷键： `Ctrl+D`

把光标放在一个单词上（或者选中一段文字），按 `Ctrl+D`（Mac 上是 `Cmd+D`）。

- 第一次按 `Ctrl+D`：VS Code 会选中当前光标所在的单词。
- 第二次按 `Ctrl+D`：VS Code 会找到下一个相同的单词，**同时选中它**。现在你有两个光标了。
- 第三次按 `Ctrl+D`：第三个相同的单词也被选中。现在你有三个光标。

这三个光标是**同时工作的**。你输入任何东西，三个光标位置会**同时**出现你输入的内容。你按删除键，三个光标位置会**同时**删除。你按方向键，三个光标会**同时**移动。

Kate 把光标放在“守夜人”上，按了五次 `Ctrl+D` ——整篇笔记里所有五个“守夜人”全部被选中。然后她敲了“守夜者”——五个地方同时改完了。她说：“我以前改这个错别字要花半分钟，一个一个找、一个一个改。现在我只需要五次 `Ctrl+D` 和三个字。这不是快捷键，这是时间机器。”

3.2.2 跳过不需要的匹配： `Ctrl+K Ctrl+D`

有时候，`Ctrl+D` 会选中一个你不想要的位置——比如“守夜者”出现了六次，但有一处是故意写成“守夜人”的，你不想改那一处。

这时候，按 `Ctrl+K` 然后 `Ctrl+D`（注意是先后按，不是同时按）。这个操作会**跳过当前匹配**，把刚刚选中的那个额外光标取消掉，然后跳到下一个匹配。

- `Ctrl+D`：选中下一个相同词，增加一个光标
- `Ctrl+K Ctrl+D`：跳过当前选中的那个词，取消刚才增加的光标

Kate 在第六个“守夜人”上不想改，按了 `Ctrl+K Ctrl+D`，那个光标消失了。她说：“所以我可以选择性跳过——不是全选，是挑着选。”

3.2.3 在任意位置添加光标： `Alt+Click`

如果你想要的光标位置不是同一个单词——比如你想在三行代码的末尾各加一个分号——`Ctrl+D` 帮不了你。

这时候用鼠标：按住 `Alt` 键（Mac 上是 `Option` 键），然后在你想加光标的位置**点一下**。每点一下，那个位置就会出现一个新光标。你可以点出任意多个光标，分布在不同行的不同列。

然后你输入任何东西，所有光标位置会同时出现。

Kate 按住 `Alt`，在三行 Markdown 标题的末尾各点了一下，然后敲了  ——三个标题末尾同时出现了一个火焰表情。她说：“这个操作是‘随心所欲版’的多光标——我想在哪加就在哪加。”

3.2.4 多光标的使用场景

场景	用什么操作	示例
批量修改同一个词	<code>Ctrl+D</code>	把“守夜人”改成“守夜者”
在多个行末尾同时加内容	按住 <code>Alt</code> + 逐行点击	在三行标题末尾同时加 

场景	用什么操作	示例
同时编辑多个相同的结构	Ctrl+D	修改多个 <a> 标签里的链接地址
格式化多个相似的列表项	Ctrl+D 或 Alt+Click	给多个列表项的开头加 **

守夜人心法： Ctrl+D 是你对付“重复劳动”的最强武器。如果你发现自己在做同一件事超过三次——停下来，想想能不能用 Ctrl+D 一口气搞定。这不是偷懒，这是效率。

🎉 3.1 和 3.2，通关！

你现在已经可以：

- ✅ 用 Ctrl+P 在几秒内打开任何文件，不需要鼠标
- ✅ 用模糊匹配和路径匹配快速定位文件
- ✅ 用 Ctrl+D 同时选中多个相同单词，批量修改
- ✅ 用 Ctrl+K Ctrl+D 跳过不需要的匹配
- ✅ 用 Alt+Click 在任意位置添加光标

Kate 把她之前写的一篇笔记里所有的“VSCode”改成了“VS Code”——用了十次 Ctrl+D，一秒改完。她说：“我以前改这个词，要全选、查找替换、确认十二次。现在只需要选中第一个、按十次 Ctrl+D、敲一个空格。我以前的十分钟，现在十秒。”

下一节，我们要学**分屏写作**——左边写源码，右边看预览，再也不用 Alt+Tab 来回切。

守夜人，继续敲！🔪

3.3 分屏写作

你正在写一篇 Markdown 笔记，左边是源码，右边是预览。这已经很爽了——但有时候，你需要的不是“左边写右边看”，而是**同时看两份不同的内容**。

比如：你正在写一篇教程，需要参考另一篇笔记里的内容。你现在的做法可能是 Ctrl+Tab 在两个文件之间来回切——看一眼参考笔记，切回来写几行，忘了，再切回去看一眼，再切回来。两个文件在你面前来回跳，思路也跟着来回跳。

分屏写作就是解决这个问题的。它让你在**同一个 VS Code 窗口里同时打开两个编辑器**，左右各一个，互相独立。你可以左边看参考笔记，右边写自己的内容——或者左边写 Markdown 源码，右边看实时预览——或者左边写代码，右边看文档。

Kate 说：“所以分屏就是‘双屏模式’。我不是在两个窗口之间切来切去，而是同时看到两个文件。”

3.3.1 打开分屏

按 `Ctrl+\\`（Mac 上是 `Cmd+\\`）。

编辑区会从中间分成两半。当前打开的文件会出现在左边，右边是一个空的编辑区。

你现在有两个编辑器并排在一起。它们互相独立——你在左边滚动，右边不动；你在右边打字，左边不变。你可以把任意文件拖到左边或右边，也可以在左边和右边各自打开不同的文件。

如果你想把两个编辑器上下排列而不是左右排列，按 `Ctrl+K` 然后 `Ctrl+\\`（注意是先按 `Ctrl+K`，松开，再按 `Ctrl+\\`）。这个操作会在**左右分屏**和**上下分屏**之间切换。

Kate 按了一下 `Ctrl+\\`，编辑区变成了两半。她说：“所以我可以左边写笔记、右边看参考文章？不用 `Alt+Tab` 来回切？”

“对。分屏是‘一边读一边写’的最佳姿势。”

3.3.2 在分屏之间切换

你现在有两个编辑器，光标停在其中一个里面。怎么把光标切到另一个编辑器里？

快捷键	作用
<code>Ctrl+1</code>	跳到 左边 的编辑器
<code>Ctrl+2</code>	跳到 右边 的编辑器

如果你开了三个分屏（是的，你可以继续按 `Ctrl+\\` 再分出一个），`Ctrl+3` 跳到第三个，以此类推。

Kate 说：“所以 `Ctrl+\\` 是开分屏，`Ctrl+1` 和 `Ctrl+2` 是在左右之间跳。三个键，两个动作，我的屏幕变成了两张桌子。”

3.3.3 关闭分屏

把不想保留的那个编辑器里的文件关掉：`Ctrl+W`。

当右边编辑器里的最后一个文件被关掉，右边的编辑器会自动消失，剩下左边一个编辑器独占整个屏幕——回到单屏模式。

如果你想把当前编辑器**最大化**（暂时盖住另一个编辑器），按 `Ctrl+Shift+Enter`。再按一次恢复分屏。这不是“关掉”另一个编辑器，只是“暂时让它让开”，让你专注当前文件。

3.3.4 分屏的实战场景

场景	左边	右边
看教程	我的教程 Markdown 源码	实时预览（MPE）
写笔记	你的笔记 Markdown 源码	参考文章/视频
写代码	你的代码文件	文档/API 参考
对比文件	旧版本文件	新版本文件

最后一个场景特别有用——你在 02 教程 1.4.4 节学过 `code -d` 命令行对比文件，分屏是同一个功能的“编辑器内版本”。你可以把两个文件分别拖到左右两个编辑器里，用眼睛逐行对比哪里改了哪里没改。

Kate 把她昨天和今天的笔记分别放在左右两个编辑器里，一行一行对比。她说：“昨天的我在左边，今天的我在右边。中间那条线，是我一天学到的东西。”

3.3.5 分屏和预览的区别

你可能已经发现了——你从 01 教程开始就在用“左边写源码，右边看预览”。那也是一种分屏。MPE 预览窗口本质上就是一个**只读的分屏编辑器**——它和普通编辑器分屏的区别只是：预览窗口不能编辑，只能看。

如果你只想一边写一边看效果，用预览就够了。如果你需要同时编辑两个文件——两个都能改、都能保存——用分屏。

Kate 说：“所以我这一个月来一直在用分屏，只是我自己不知道。左边写 Markdown，右边 MPE 预览——这就是分屏。只不过右边的编辑器是只读的。”

“对。分屏是你已经在用的功能，这一节只是告诉你——右边的编辑器不只能预览，还能打开任何文件。”

3.3.6 本节小结

快捷键	作用
Ctrl+\ <code></code>	打开/切换分屏（左右）
Ctrl+K Ctrl+\ <code></code>	切换分屏方向（左右 ↔ 上下）
Ctrl+1 / Ctrl+2	跳到左边/右边的编辑器
Ctrl+W	关闭当前编辑器（分屏也随之关闭）

快捷键	作用
Ctrl+Shift+Enter	最大化/还原当前编辑器

分屏不是“高级技巧”——它是你每天都会用到的写作方式。左边读、右边写、中间一条线，你的思路在这条线上来回流动，不再被 `Alt+Tab` 打断。✍

3.4 保存时自动格式化

什么是格式化文件？

格式化文件，就是让源码的“长相”保持一致。同一篇笔记，你可以写成这样：

```
## 今天学了什么
```

- 标题
- 列表
- 表格

也可以写成这样：

```
## 今天学了什么
```

- 标题
- 列表
- 表格

两段源码在预览里渲染出来的效果**完全一样**——都是一个大标题下面跟着三个小黑点。但你扫一眼源码就知道，第一段整齐、干净、每个 `-` 后面都有空格；第二段有的有空格有的没有，读起来像走在坑坑洼洼的路上。

格式化做的事情就是：**把第二段自动变成第一段**。它不改你写的内容，只改你写的方式——空格、缩进、空行、对齐。这些细节不影响电脑理解你的文档，但影响人阅读你的源码。

为什么需要自动格式化？

你可以手动遵守这些规则——01 教程 4.8 节教了你所有排版技巧，你完全可以每次写完笔记之后自己检查一遍。但三个月后你回来翻一篇旧笔记，你会发现自己当初漏了几个空格、忘了几处空行。人不是机器——人会忘、会累、会在凌晨一点写完笔记之后懒得检查。

自动格式化就是帮你解决这个问题的：你只管写内容，格式交给工具。按一下保存，源码自动变整齐。

Kate 说：“所以自动格式化就是‘我写的时候随便乱写，保存的时候它帮我收拾’。就像一个自动整理桌面的机器人。”

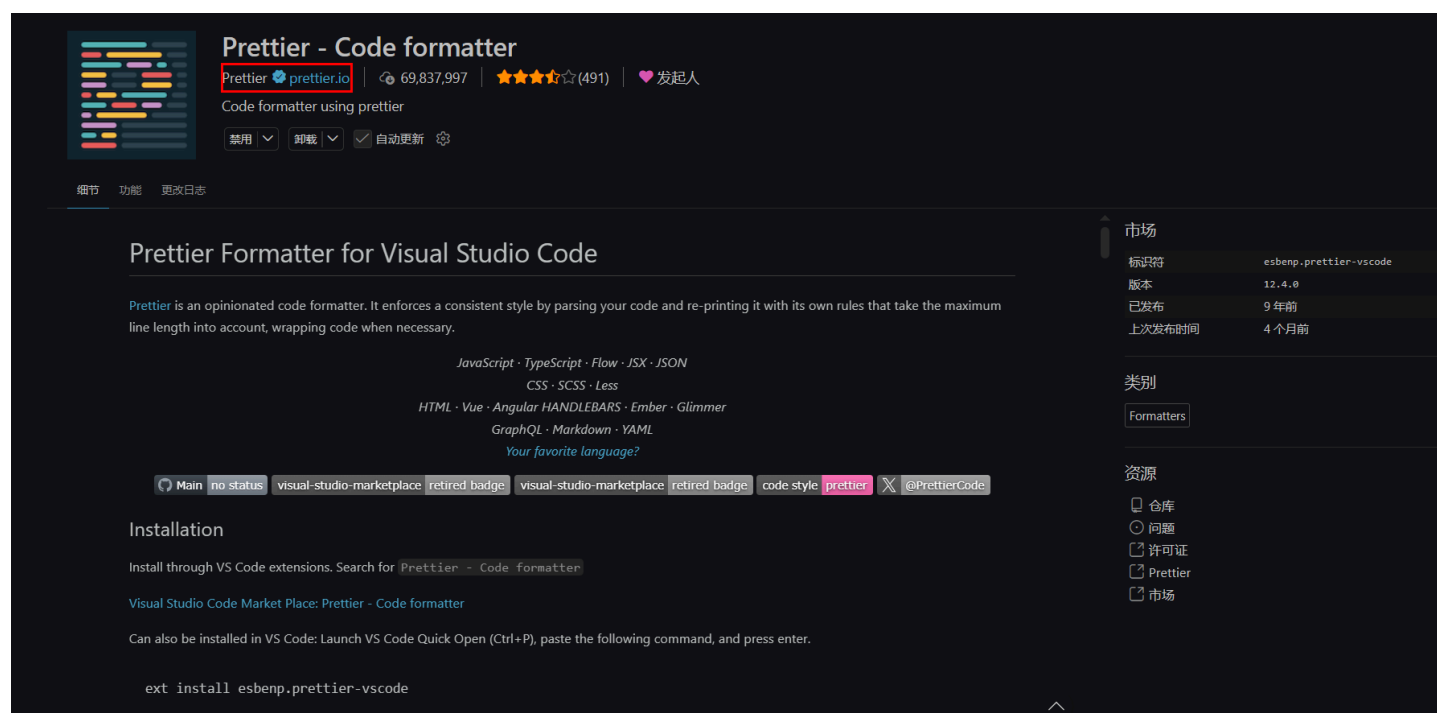
3.4.1 配置自动格式化

自动格式化不是 VS Code 自带的功能——它需要你安装一个额外的插件：**Prettier**。

Prettier 是一个通用的代码格式化工具，支持 Markdown、HTML、CSS、JavaScript 等几十种语言。你在 01 教程里装的六神装没有它，因为 01 教程的定位是“写 Markdown 的语法”，而 Prettier 是“帮你维护代码整洁的工具”。现在你在 02 教程里学 VS Code 的效率技巧，Prettier 正好属于这一块。

第一步：安装 Prettier 插件

1. 按 `Ctrl+Shift+X` 打开扩展商店
2. 搜索 `Prettier - Code formatter`
3. 认准作者是 **Prettier**（不是其他个人开发者），安装它



第二步：开启保存时自动格式化

1. 按 `Ctrl+,` 打开 VS Code 设置
2. 在搜索框里输入 `format on save`
3. 找到 **Editor: Format On Save** 选项，**勾上它**

这个开关打开之后，VS Code 会在你保存文件时自动调用格式化工具——但具体用哪个工具来格式化 Markdown 文件，还需要下一步来指定。

第三步：把 Prettier 设为 Markdown 文件的默认格式化工具

1. 按 `Ctrl+,` 打开 VS Code 设置
2. 在搜索框里输入 `default formatter`
3. 找到 **Editor: Default Formatter** 选项，点击它
4. 在弹出的下拉列表里，选择 **Prettier - Code formatter**（作者是 Prettier）

这样设置之后，VS Code 就知道：“所有支持的文件类型，优先用 Prettier 来格式化。”从现在开始，你每次按 `Ctrl+S` 保存 Markdown 文件，Prettier 会自动帮你格式化。

Kate 按完这三步，打开了一篇之前写的笔记，按了 `Ctrl+S`。她看着屏幕上那些歪歪扭扭的表格竖线自动归位，多余的空行自动消失，说：“这就跟我妈收拾我的房间一样——我什么都没干，房间就干净了。”

3.4.2 自动格式化做了什么？

你保存文件的时候，Prettier 在背后帮你做了这些事：

你写的时候	保存后 Prettier 帮你
行尾多打了几个空格	自动删掉多余空格
表格竖线歪歪扭扭	自动对齐竖线
列表缩进用了 Tab	自动转成空格（保持一致）
代码块前后少了空行	自动补上空行
文件末尾少了一个空行	自动在最后加一行

这些都是你在 01 教程 4.8 节“排版技巧”里学过的规则——当时你是手动遵守的，现在是 Prettier 自动帮你遵守。你不需要记住每一条规则，只需要记住一个动作：**写完按 `Ctrl+S`**。

Kate 说：“所以我学了那么多排版规则——空格、空行、缩进、对齐——现在一个 `Ctrl+S` 全搞定了？”

“对。01 教程教你‘规则是什么’，02 教程教你‘让工具替你执行规则’。先知道为什么，再学会怎么自动化。顺序不能反——如果你不知道 Prettier 在背后做了什么，它帮你改了些什么你都不知道。”

3.4.3 Prettier 和 Markdown Lint 的区别

你可能想问：“自动格式化和 Markdown Lint 有什么区别？它们不都是帮我检查 Markdown 的吗？”

工具	干什么的	什么时候工作
Markdown Lint	检查你写错的地方，画波浪线提醒你	你 正在写 的时候，实时检查
Prettier	自动修正格式问题，帮你整理源码	你 保存文件 的时候，自动执行

它们不是竞争对手——它们是搭档。Markdown Lint 是教官，在你写的时候站在你身后，看到哪里不对就“啧——”一声，画一道波浪线。Prettier 是勤务兵，你按保存的时候默默帮你把东西收拾整齐。教官提醒

你哪里错了，勤务兵帮你改掉。

Kate 说：“所以我写的时候 Lint 教官在我耳边嘖，保存的时候 Prettier 帮我收拾残局。这俩人一个唱红脸一个唱白脸。”

3.4.4 如果你不想格式化某个文件

Prettier 默认会对所有支持的文件类型生效。但有时候你不想让它格式化某个文件——比如你故意写了一段格式“混乱”的代码来展示错误示例。

在项目的根目录（最外层文件夹）创建一个 `.prettierignore` 文件，内容写法跟 `.gitignore` 一样：

```
# 不格式化这些文件
演示-错误示范.md
测试/乱码笔记.md
```

Prettier 看到这个文件，会跳过你列出的那些文件，不动它们。

3.4.5 本节小结

操作	快捷键 / 方法
安装 Prettier	Ctrl+Shift+X → 搜索 Prettier - Code formatter
开启保存时自动格式化	Ctrl+, → 搜 format on save → 勾上
手动格式化当前文件	Ctrl+Shift+P → Format Document
把 Prettier 设为默认	Ctrl+Shift+P → Format Document With... → 选 Prettier
排除某个文件不格式化	创建 .prettierignore，写入文件名

自动格式化不是“炫技”——它是让你把精力集中在内容上，而不是格式上。你写的内容才是笔记的灵魂，空格和缩进是勤务兵的事。🔪

3.5 其他实用设置

前四节你学了快速打开文件、多光标编辑、分屏写作、自动格式化——这些是“每天都在用”的核心技巧。这一节我们收个尾，讲几个不用每天用、但需要的时候能救命的实用设置。

3.5.1 行号

编辑器左边那排数字就是行号。它们不是装饰——你在 1.4.3 节学过 `code -g` 跳转到指定行的时候，靠的就是行号。你在 Markdown Lint 报错的时候，它告诉你的也是“第几行有问题”。

行号默认是开的。如果你不小心关掉了（或者想确认它是开的），按 `Ctrl+`，搜索 `line numbers`，确保 **Editor: Line Numbers** 是 `on`。

3.5.2 缩进参考线

写嵌套列表或嵌套代码块的时候，你经常需要知道“当前这一行缩进了多少层”。如果你不想一个一个空格去数，可以让 VS Code 帮你画出缩进参考线——每缩进一层，就多一条淡淡的竖线。

按 `Ctrl+`，搜索 `render indent guides`，找到 **Editor: Render Indent Guides**，确保它是 `on`（默认就是开的）。如果你想让参考线更明显，把 **Editor: Guides: Indentation** 改成 `always`。

Kate 打开了缩进参考线，看着编辑器里那些淡淡的竖线，说：“原来我的列表嵌套有这么多层。不用数空格了，数竖线就行。”

3.5.3 迷你地图

VS Code 右侧有一长条缩略图，把你整个文件的内容压缩成一个微缩视图。这叫**迷你地图**。它的作用是让你快速定位——你看到迷你地图上某个区域颜色比较深（说明那里代码很密集），点一下，编辑区就跳到那个位置。

对于写代码的人来说，迷你地图很有用。但对于写 Markdown 的人来说，迷你地图大多数时候只是占位置。你的 Markdown 文件通常不会长到需要迷你地图来导航——目录和 `Ctrl+P` 已经够用了。

如果你觉得迷你地图碍眼，按 `Ctrl+`，搜索 `minimap`，找到 **Editor: Minimap Enabled**，把它关掉。编辑区会多出一小截空间——你的文字多了一行呼吸。

Kate 关掉了迷你地图，说：“我从来没有用过这个东西。它在右边一直待着，像一个我从来不去逛的景区。”

3.5.4 保存时自动删除行尾空格

你在 01 教程 3.4 节学过——Markdown 的段内换行需要在行尾敲两个空格。但 VS Code 有一个“好心”的设计：默认情况下，保存文件的时候它会自动删掉所有行尾的多余空格，包括你辛辛苦苦敲的那两个用来换行的空格。

结果就是：你明明敲了两个空格，预览里换行了，保存之后换行消失。你还以为自己哪里写错了，其实是 VS Code 把你的空格当“多余的”给清理了。

如果你经常用段内换行（两个空格 + 回车），建议把这个自动清理关掉。按 `Ctrl+,`，搜索 `trim trailing whitespace`，找到 **Files: Trim Trailing Whitespace**，**取消勾选**。如果你只想对 Markdown 文件关掉这个功能，可以在设置里搜 `files.trimTrailingWhitespace`，然后在 Markdown 的专属设置里单独关闭它。

Kate 看着设置页面，说：“所以 VS Code 会偷偷删掉我的空格——那些我故意敲的空格。这不是帮我，是在坑我。”

“对。这是一个‘好心办坏事’的经典案例。你知道它的存在之后，就可以关掉它，或者利用它——取决于你写的是什么文件。”

3.5.5 本节小结

你想干什么	怎么设置
显示行号	<code>Ctrl+,</code> → 搜 <code>line numbers</code> → 确保 <code>on</code>
显示缩进参考线	<code>Ctrl+,</code> → 搜 <code>render indent guides</code> → 确保 <code>on</code>
关掉迷你地图	<code>Ctrl+,</code> → 搜 <code>minimap</code> → 取消勾选
保留行尾空格（Markdown 换行用）	<code>Ctrl+,</code> → 搜 <code>trim trailing whitespace</code> → 取消勾选

这些设置不需要全记住。你现在只需要知道：**VS Code 几乎什么都能调**。以后你觉得哪里不舒服——字体太小、主题太暗、迷你地图挡视线——先别忍着，去设置里搜一下，大概率能调。调完之后的 VS Code，不是“默认的那个 VS Code”，是**你的 VS Code**。🔧

第四章 - 快捷键：从常用到速查

前几章你学了 VS Code 的安装、界面布局、写作效率技巧。这些章节里散落着很多快捷键——`Ctrl+P` 快速打开文件、`Ctrl+D` 多光标编辑、`Ctrl+\` 分屏写作、`Ctrl+S` 保存时自动格式化。

这一章不是教你新快捷键——是把前面散落的快捷键**收拢到一起**，按使用频率排序，让你一眼就能找到自己需要的那个。

Kate 说：“所以这一章不是‘学新的’，是‘复习旧的’。把我已经会的东西整理成一张表。”

“对。而且不只是表——前五节我会挑五个最常用的快捷键，单独拆开讲透。不是‘这个键干什么’，而是‘你什么时候会用到它、用错了会怎样、怎么和别的快捷键配合’。”

4.1 Ctrl+P：快速打开文件

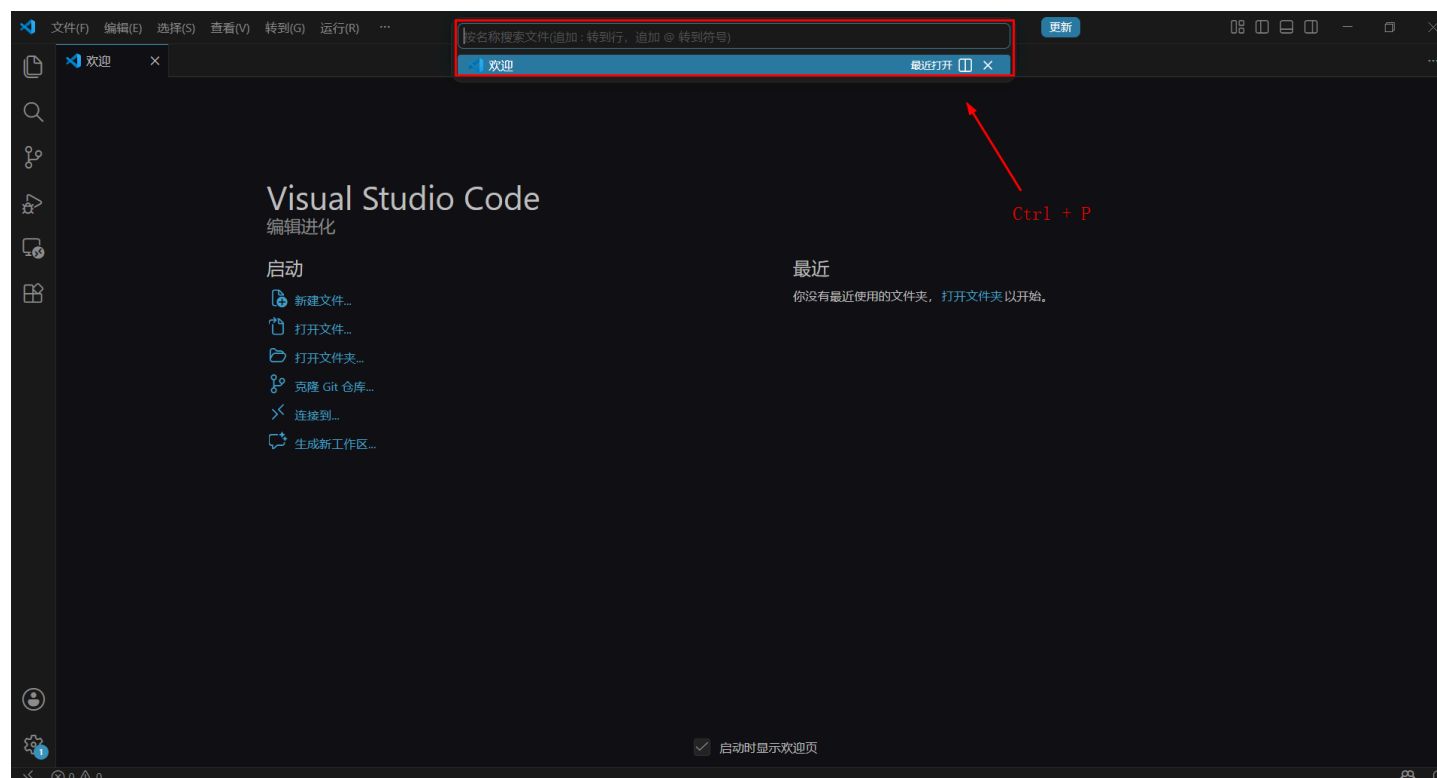
你在 VS Code 里是怎么打开文件的？

是不是这样：左手离开键盘 → 摸到鼠标 → 眼睛看向左侧的资源管理器 → 在一堆文件夹里一层一层地找 → 点一下文件名 → 手回到键盘上。如果你同时开着七八个文件，这个动作你一天要重复几十次。每一次都在打断你的写作思路。

`Ctrl+P` 就是要消灭这个“切换成本”。它是你在 VS Code 里用的最多的快捷键——不是之一。

4.1.1 怎么用

按 `Ctrl+P`（Mac 上是 `Cmd+P`），屏幕顶部中央会弹出一个搜索框。



在这个搜索框里，输入文件名的一部分——不需要完整的文件名，不需要完整的路径。只需要几个字母，VS Code 会模糊匹配所有你当前打开过的文件。

比如你想打开 `01-玩转-Markdown.md`，你只需要输入 `01` 或 `mark` 或 `md` 中的任意几个字母，文件名就会出现在列表里。按回车，文件立刻打开。

Kate 按了 `Ctrl+P`，输入了 `mark`，她的 Markdown 教程文件瞬间出现在列表最前面。她按了回车，文件打开了。她说：“我以前每次打开这个文件，都是去侧边栏翻——先找到文件夹，再找到子文件夹，再在一堆文件里找。现在只需要 `Ctrl+P` 然后打几个字母。这中间我省了多少秒？”

“每次大概 3-5 秒。一天几十次，就是几分钟。一年下来，可能是几十个小时——几十个小时你不去找文件，而是在写内容。”

4.1.2 搜索技巧

`Ctrl+P` 不只是“输入文件名”，它有几个非常实用的搜索技巧：

技巧	输入方式	作用	示例
模糊匹配	文件名里的任意几个字母	快速缩小列表范围	输入 <code>pla</code> 能匹配到 <code>01-玩转-Markdown.md</code>
路径匹配	文件夹名 + 文件名	在特定文件夹里找文件	输入 <code>chapter/01</code> 只匹配 <code>chapter</code> 文件夹里的 <code>01</code> 文件
最近打开	直接按 <code>Ctrl+P</code> ，不看列表	列表默认按最近打开排序	你刚关掉的文件排在最上面，直接回车就打开

模糊匹配的规则： `Ctrl+P` 的搜索不是“精确匹配”，而是“模糊匹配”。你输入 `vm`，它可能匹配到 `VIM-从入门到得休.md`，因为文件名里有 `V` 和 `M` 这两个字母——即使它们不连在一起。你输入的字母不需要在文件名里连续出现，只要文件名里有这些字母，VS Code 就能找到它。

这意味着你可以用最少的按键找到目标文件。`01` 匹配所有以 `01` 开头的文件，`mark` 匹配所有文件名里有“mark”的文件，`css` 匹配所有 CSS 文件。你不需要记住完整的文件名——只需要记住文件名的“特征”。

路径匹配的高级用法：如果你的项目里有两个同名文件——比如 `README.md` 同时出现在根目录和 `docs/` 文件夹里——你可以用路径来区分。输入 `docs/readme`，列表里只会显示 `docs/README.md`。输入 `root/readme` 或直接输入 `readme`，VS Code 会列出所有匹配的文件，你手动选一个。

4.1.3 和其他打开方式的对比

方式	适合场景	不适合场景
<code>Ctrl+P</code>	知道文件叫什么，想快速跳过去	不知道文件叫什么，想浏览整个项目结构
侧边栏	不知道文件叫什么，想一层层展开找	知道文件叫什么——用侧边栏翻太慢了
<code>Ctrl+Tab</code>	在最近打开的两三个文件之间切换	想打开一个很久没打开的文件
<code>code .</code>	刚打开 VS Code，想打开整个项目	已经在 VS Code 里了，只想打开一个文件

这四种方式不是“谁替代谁”——它们是互补的。知道文件名就用 `Ctrl+P`，不知道就用侧边栏，在两个文件之间来回切用 `Ctrl+Tab`，刚启动 VS Code 用 `code .`。

Kate 说：“所以我不用在侧边栏和 `Ctrl+P` 之间二选一——知道名字用 `Ctrl+P`，不知道名字用侧边栏。它们不是竞争对手，是队友。”

4.1.4 特殊用法：用 `Ctrl+P` 执行命令

`Ctrl+P` 的搜索框不只是用来打开文件的。你在输入框里输入 `>`（一个大于号），它会立刻切换到**命令模式**——和 `Ctrl+Shift+P` 打开的命令面板完全一样。输入 `@`，它会列出当前文件里所有的标题和函数——你可以直接跳转到某个标题。输入 `:`，你可以输入行号直接跳转到那一行。

输入	作用	示例
文件名	打开文件	01-markdown
> 命令名	执行命令（等同于命令面板）	> format document
@ 标题名	跳转到当前文件的某个标题	@ 分屏写作
: 行号	跳转到当前文件的某一行	:42

这四个用法里，你最常用的是第一个——打开文件。但后面三个值得记住：它们让你不需要再按 `Ctrl+Shift+P`、`Ctrl+Shift+O`、`Ctrl+G` ——所有的跳转和命令都在同一个搜索框里完成。

Kate 在 `Ctrl+P` 里输入了 `:42`，光标直接跳到了当前文件的第 42 行。她说：“所以 `Ctrl+P` 不只是‘打开文件’，是一个‘全能搜索框’。文件、命令、标题、行号——我只需要记住这一个快捷键。”

“精准。`Ctrl+P` 是 VS Code 的万能入口。你用的越多，越不需要记住其他快捷键。”

4.1.5 养成肌肉记忆

`Ctrl+P` 是你一天里用得最多的快捷键。把它刻进肌肉记忆——每当你需要打开另一个文件，第一反应不是去摸鼠标，而是按下 `Ctrl+P`。

如果你觉得“我老是忘记用 `Ctrl+P`”，试试这个办法：下一次你想打开文件的时候，先把手从鼠标上拿开，放在键盘上，然后问自己：“我要打开的那个文件，名字里有什么字母？”输入那几个字母，回车。重复一周，你的手指会自动去找 `Ctrl+P`。🔪

4.2.2 你最常用的几个命令

以下是你写 Markdown 时最可能用到的命令：

输入关键词	找到的命令	作用
table of contents	Markdown All in One: Create Table of Contents	自动生成目录
format document	Format Document	用 Prettier 格式化当前文件
export to	Markdown Preview Enhanced: Export to...	导出 PDF/HTML/PNG
toggle preview	Markdown: Open Preview to the Side	打开实时预览
format all tables	Markdown Table: Format All Tables	一键对齐所有表格
fix all	Markdownlint: Fix all supported violations	自动修复所有 Lint 报错
settings	Preferences: Open Settings	打开设置页面
keyboard shortcuts	Preferences: Open Keyboard Shortcuts	查看所有快捷键

你不需要记住这些命令的全名——只需要记住关键词。想生成目录就输入 `toc` 或 `table of`，想格式化就输入 `format`，想导出就输入 `export`。命令面板是模糊匹配的——你输入的字母只要在命令名里存在（不一定连续），它就能找到。

Kate 说：“所以我以后遇到‘这个功能到底有没有快捷键’的时候，不需要去 Google 了——直接 `Ctrl+Shift+P` 搜关键词。搜不到，说明 VS Code 没这个功能。搜到了，直接回车执行。”

两个你不知道但很实用的命令：

输入关键词	找到的命令	作用
reopen closed	View: Reopen Closed Editor	重新打开你刚才关掉的文件
collapse all	Fold All	折叠所有标题块，只看文档大纲

第一个命令救了 Kate 一命：她有一次不小心关掉了写了一半的笔记，`Ctrl+Z` 不管用（撤销只针对编辑器内的文字操作，不针对文件关闭），急得差点摔键盘。后来她用 `Ctrl+Shift+P` → `reopen closed` 找了回来。这个命令可以重复执行——按一次打开最近关掉的一个文件，再按一次打开倒数第二个。

第二个命令适合长文档：你在写一篇几千字的教程，想在各个章节之间快速跳转。用 `Fold All` 把全文折叠成只有标题，找到你要去的那一节，点一下展开，立刻到达。

4.2.3 命令面板和 Ctrl+P 的关系

你在 4.1.4 节学过——在 `Ctrl+P` 的搜索框里输入 `>`，它会立刻变成命令面板。两者的区别在于：

入口	默认搜索	但也可以
<code>Ctrl+P</code>	搜索文件名	输入 <code>></code> 后变成命令面板
<code>Ctrl+Shift+P</code>	搜索命令	只能搜索命令

如果你更习惯“一个入口做所有事”，可以只用 `Ctrl+P` ——打开文件直接输入文件名，执行命令先输入 `>` 再输入命令名。如果你更喜欢“打开文件”和“执行命令”用两个不同的快捷键，那就分别用 `Ctrl+P` 和 `Ctrl+Shift+P`。两种方式都可以，选你顺手的那种。

Kate 说：“所以我其实不需要 `Ctrl+Shift+P` ——我只需要 `Ctrl+P`，然后多打一个 `>`。但两个都记着也没什么坏处。”

“对。 `Ctrl+Shift+P` 少按一个字符， `Ctrl+P` 更通用。选哪个取决于你今天有多懒。”

4.2.4 养成肌肉记忆

`Ctrl+Shift+P` 是 VS Code 里最重要的快捷键之一——不是因为它能做什么，而是因为它能找到所有你会忘记快捷键的功能。

每次你想做一件事但不知道快捷键的时候，别去菜单栏翻，别去 Google 搜。先按 `Ctrl+Shift+P`，输入你想做的事的关键词。如果命令存在，它会出现——同时它旁边还会显示这个命令的快捷键（如果有的话）。你不仅完成了操作，还顺便学会了它的快捷键。下次你就可以直接用快捷键，不用再打开命令面板。

这就是“用命令面板学快捷键”的循环：忘记快捷键 → 打开命令面板 → 执行命令（同时看到快捷键） → 下次直接用快捷键。反复几轮，你不需要刻意背快捷键——你的手指会自动记住。✍

4.3 Ctrl+D：选中下一个相同词

你有没有遇到过这种情况：一篇笔记里，你写错了同一个词——比如把“守夜者”写成了“守夜人”——这个词出现了五六次。你一个一个个地找到、选中、修改、再找下一个。改了五次，手酸眼累。

`Ctrl+D` 就是要让你**同时改完所有这些地方，一次搞定**。

4.3.1 怎么用

把光标放在一个单词上（或者选中一段文字），按 `Ctrl+D`（Mac 上是 `Cmd+D`）。

- 第一次按 `Ctrl+D`：VS Code 会选中当前光标所在的单词。
- 第二次按 `Ctrl+D`：VS Code 会找到下一个相同的单词，**同时选中它**。现在你有两个光标了。
- 第三次按 `Ctrl+D`：第三个相同的单词也被选中。现在你有三个光标。

这三个光标是**同时工作的**。你输入任何东西，三个光标位置会**同时**出现你输入的内容。你按删除键，三个光标位置会**同时**删除。你按方向键，三个光标会**同时**移动。

Kate 把光标放在“守夜人”上，按了五次 `Ctrl+D` ——整篇笔记里所有五个“守夜人”全部被选中。然后她敲了“守夜者”——五个地方同时改完了。她说：“我以前改这个错别字要花半分钟，一个一个找、一个一个改。现在我只需要五次 `Ctrl+D` 和三个字。这不是快捷键，这是时间机器。”

4.3.2 跳过不需要的匹配

有时候，`Ctrl+D` 会选中一个你不想要的位置——比如“守夜者”出现了六次，但有一处是故意写成“守夜人”的，你不想改那一处。

这时候，按 `Ctrl+K` 然后 `Ctrl+D`（注意是先后按，不是同时按）。这个操作会**跳过当前匹配**，把刚刚选中的那个额外光标取消掉，然后跳到下一个匹配。

操作	作用
<code>Ctrl+D</code>	选中下一个相同词，增加一个光标
<code>Ctrl+K Ctrl+D</code>	跳过当前选中的那个词，取消刚才增加的光标

Kate 在第六个“守夜人”上不想改，按了 `Ctrl+K Ctrl+D`，那个光标消失了。她说：“所以我可以选择性跳过——不是全选，是挑着选。”

4.3.3 在任意位置添加光标

如果你想要的光标位置不是同一个单词——比如你想在三行代码的末尾各加一个分号——`Ctrl+D` 帮不了你。

这时候用鼠标：按住 `Alt` 键（Mac 上是 `Option` 键），然后在你想加光标的位置**点一下**。每点一下，那个位置就会出现一个新光标。你可以点出任意多个光标，分布在不同行的不同列。

然后你输入任何东西，所有光标位置会同时出现。

Kate 按住 `Alt`，在三行 Markdown 标题的末尾各点了一下，然后敲了  ——三个标题末尾同时出现了一个火焰表情。她说：“这个操作是‘随心所欲版’的多光标——我想在哪加就在哪加。”

4.3.4 多光标的使用场景

场景	用什么操作	示例
批量修改同一个词	<code>Ctrl+D</code>	把“守夜人”改成“守夜者”
在多个行末尾同时加内容	按住 <code>Alt</code> + 逐行点击	在三行标题末尾同时加 
同时编辑多个相同的结构	<code>Ctrl+D</code>	修改多个 <code><a></code> 标签里的链接地址
格式化多个相似的列表项	<code>Ctrl+D</code> 或 <code>Alt+Click</code>	给多个列表项的开头加 <code>**</code>

守夜人心法： `Ctrl+D` 是你对付“重复劳动”的最强武器。如果你发现自己在做同一件事超过三次——停下来，想想能不能用 `Ctrl+D` 一口气搞定。这不是偷懒，这是效率。

4.3.5 养成肌肉记忆

`Ctrl+D` 和 `Ctrl+K Ctrl+D` 是一对搭档——一个加光标，一个取消光标。它们配合使用，让你可以精确控制哪些地方改、哪些地方不改。

下次你在笔记里发现一个写错的词，别一个一个手动改。把光标放在它上面，按 `Ctrl+D` 直到所有想改的地方都被选中，跳过不想改的地方用 `Ctrl+K Ctrl+D`，然后输入正确的词。全部过程不超过十秒。

Kate 把她之前写的一篇笔记里所有的“VSCode”改成了“VS Code”——用了十次 `Ctrl+D`，一秒改完。她说：“我以前改这个词，要全选、查找替换、确认十二次。现在只需要选中第一个、按十次 `Ctrl+D`、敲一个空格。我以前的十分钟，现在十秒。”

4.4 Ctrl+`：打开/关闭终端

你在 02 教程第二章学过——VS Code 底部有一个集成终端，它就是一个嵌入在编辑器里的命令行窗口。你不用切换窗口，在编辑器里就能敲命令。

打开和关闭终端只需要一个快捷键：`Ctrl+``（键盘左上角 `ESC` 下面的那个反引号键，Mac 上是 `Cmd+``）。

4.4.1 怎么用

按一下 `Ctrl+``，终端面板从底部弹出。再按一下，终端面板缩回去。

终端弹出来之后，光标会自动定位到终端的输入行里。你可以直接输入命令，不需要再用鼠标点一下终端区域。

Kate 按了 `Ctrl+``，底部弹出一个终端面板。她输入 `ls`，看到了当前文件夹里的所有文件。她说：“原来我不用最小化 VS Code 去开 PowerShell——它自己就有。这就是你 02 前言里说的‘指挥中心’。”

4.4.2 你在终端里最常做的事

命令	作用	什么时候用
<code>code .</code>	用 VS Code 打开当前目录	在终端里切到某个文件夹，想用 VS Code 打开它
<code>git status</code>	查看 Git 状态	看看改了哪些文件
<code>git add . && git commit -m "..."</code>	提交代码	保存修改到本地仓库
<code>git push</code>	推送到 GitHub	把本地提交同步到远程仓库
<code>python 脚本名.py</code>	运行 Python 脚本	测试你写的安全工具
<code>ls / dir</code>	查看当前目录下的文件列表	看看文件夹里有什么

你在终端里输入的命令和你在系统终端里输入的完全一样——只是你现在不需要离开 VS Code 去另一个窗口。

4.4.3 终端和编辑器的切换

你正在写 Markdown，需要去终端敲一个命令。你按 `Ctrl+``，终端弹出来，敲完命令，想回到编辑器继续写。怎么回去？

快捷键	作用
<code>Ctrl+`</code>	打开/关闭终端
<code>Ctrl+1</code>	光标跳回编辑器（第一个编辑区）
<code>Ctrl+2</code>	光标跳到第二个编辑区（如果你开了分屏）

所以完整的操作流程是：`Ctrl+`` 打开终端 → 敲命令 → `Ctrl+1` 回到编辑器。三个键，来回切换不到一秒。

Kate 说：“所以 `Ctrl+`` 是‘去指挥中心’，`Ctrl+1` 是‘回办公室’。我不需要在键盘和鼠标之间反复切换，手指一直在键盘上。”

4.4.4 多个终端

你可以同时打开多个终端——就像浏览器可以同时打开多个标签页一样。

操作	快捷键	作用
新建终端	<code>Ctrl + Shift + `</code>	在当前窗口里再开一个终端
在终端之间切换	点击终端面板右上角的终端标签	切换到另一个终端
关闭当前终端	在终端里输入 <code>exit</code> 回车	关掉当前终端

什么时候需要多个终端？比如你在学习 05 教程的实战章节，一边用终端 A 运行 Python 脚本测试漏洞，一边用终端 B 查看服务器日志。两个终端互不干扰，各干各的。

Kate 开了两个终端，左边跑着 Python 脚本，右边监控着日志输出。她说：“这就像有两个助手，一个帮我干活，一个帮我望风。”

4.4.5 终端里选中和复制

终端里不能用 `Ctrl+C` 复制——因为 `Ctrl+C` 在命令行里的含义是“终止当前正在运行的程序”。在终端里复制粘贴用另一套快捷键：

操作	快捷键
复制	<code>Ctrl+Shift+C</code>
粘贴	<code>Ctrl+Shift+V</code>

如果你觉得 `Ctrl+Shift+C` 和 `Ctrl+Shift+V` 太别扭——可以在 VS Code 设置里搜 `terminal copy selection`，找到 **Terminal > Integrated: Copy On Selection**，勾上它。之后你在终端里选中任何文字，它会自动复制到剪贴板——不需要再按任何键。粘贴仍然用 `Ctrl+Shift+V`。

4.4.6 养成肌肉记忆

`Ctrl+`` 是你和终端之间的“传送门”。你需要终端的时候按一下，终端弹出来；用完再按一下，终端缩回去。你不需要在 VS Code 和系统终端之间来回切窗口——所有操作都在同一个窗口里完成。

下次你需要敲 Git 命令、运行 Python 脚本、或者用 `code .` 打开另一个项目的时候，别去开新的 PowerShell 窗口。按 `Ctrl+``，在 VS Code 里完成一切。🔗

4.5 Alt+↑/↓：整行移动

你写了一行文字，发现它应该放在上一行前面。你现在的做法可能是：选中这一整行 → `Ctrl+X` 剪切 → 光标移到上一行前面 → 回车 → `Ctrl+V` 粘贴。四步。如果这一行很长，光选中就要花好几秒。

`Alt+↑` 和 `Alt+↓` 把这个操作压缩成了一步。

4.5.1 怎么用

快捷键	作用
<code>Alt+↑</code>	把当前行 向上 移动一行
<code>Alt+↓</code>	把当前行 向下 移动一行

不需要选中整行。光标只要在这一行的任意位置，按 `Alt+↑`，整行就会和上一行交换位置。按一次移动一行，按住不放可以连续移动——这一行会一直往你按的方向挪，直到你松开键。

Mac 上是 `Option+↑` 和 `Option+↓`。

Kate 写了一段笔记，发现第三行应该放在第一行前面。她把光标移到第三行，按住 `Alt+↑` 不松手——第三行像坐电梯一样升到了第一行前面。她说：“我以前做这件事要四步：全选、剪切、找位置、粘贴。现在只需要一个动作——按住，松手。我以前的四步，现在一步。”

4.5.2 多行同时移动

如果你要移动的不止一行——比如你想把三个连续的段落整体往上移——先选中这三行，再按 `Alt+↑` 或 `Alt+↓`。三行会作为一个整体同时移动，不会散开。

选中多行不需要全选每一行——只需要确保选中区域覆盖了所有目标行的任意部分。把光标放在第一行开头，按住 `Shift` 点一下第三行末尾，这三行就被选中了。然后 `Alt+↑`，整个选中的块一起向上移动。

4.5.3 和剪切粘贴的区别

方式	步骤	适合场景
Alt+↑/↓	1 步	在当前视野内上下移动，目标位置离得不远
Ctrl+X + Ctrl+V	4 步	移动到很远的位置（比如从文件末尾移到文件开头）

如果目标位置就在附近——往上或往下几行——用 Alt+↑/↓。如果目标位置很远——比如要把文件末尾的一句话移到文件开头——用 Ctrl+X 剪切，然后 gg 跳到开头，Ctrl+V 粘贴。两种方式互补，不是替代。

4.5.4 实战场景

场景	操作
调整列表顺序	选中列表项，Alt+↑/↓ 上下调整
调整笔记段落顺序	把光标放在段落任意位置，Alt+↑/↓ 整段移动
调整代码行顺序	把光标放在那行代码上，Alt+↑/↓ 移动
批量移动多个段落	选中多个段落，Alt+↑/↓ 整体移动

Kate 把她笔记里的待办清单重新排了序——按住 Alt，上下箭头，三秒钟搞定。她说：“这个快捷键我应该第一天就学的。”

4.5.5 养成肌肉记忆

Alt+↑/↓ 是那种“用一次就回不去”的快捷键。在你还没学会它之前，移动一行文字需要四步——你习惯了，不觉得麻烦。一旦你学会了它，再回去用剪切粘贴会觉得“怎么这么慢”。

下次你想把一行文字挪到上面或下面，别选中、别剪切、别粘贴。把光标放在那一行上，按住 Alt，点一下箭头，松手。你的手指会记住这种感觉。✍

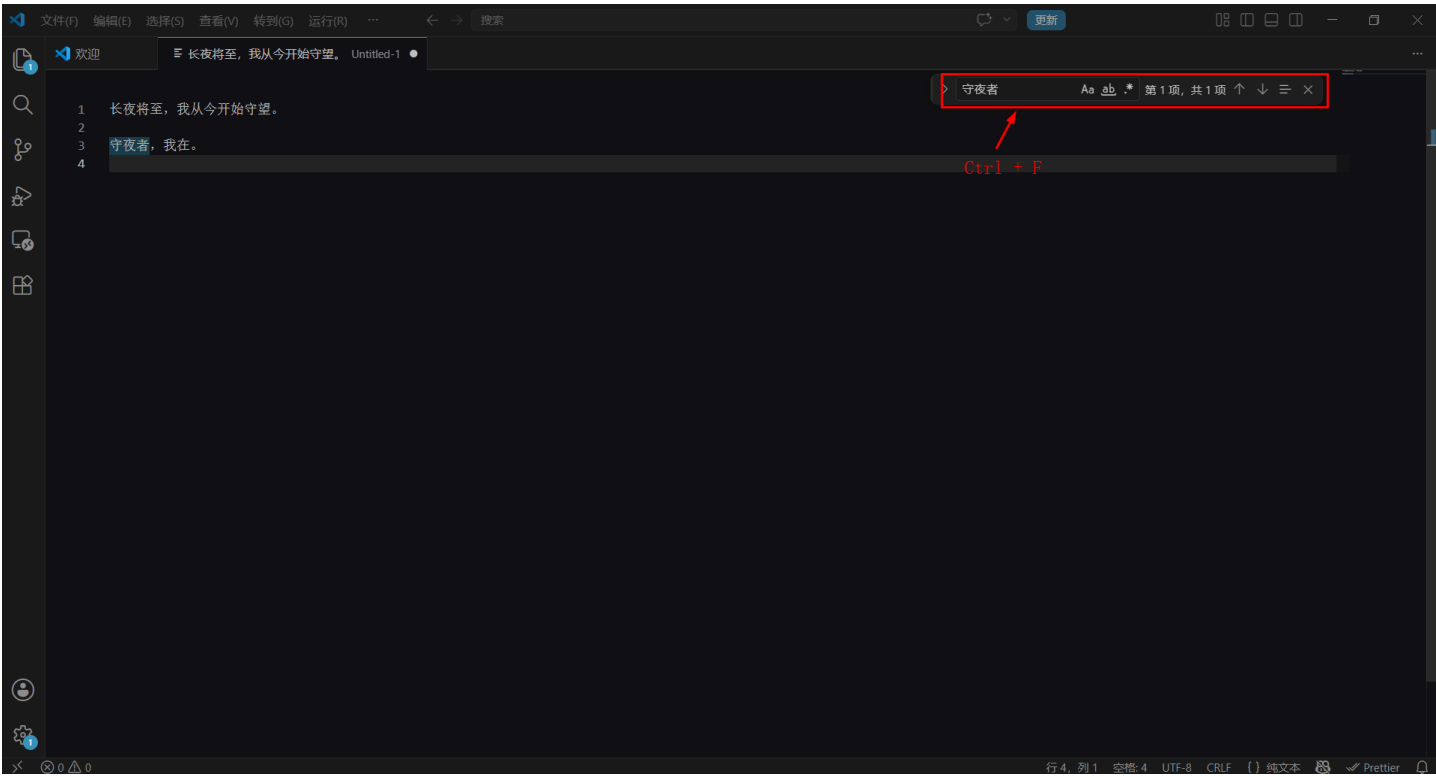
4.6 替换与查找

你写了一篇长笔记，里面把“GitHub”全写成了“Github”——小写的h。你知道错了，但你不知道这个词出现了多少次、在哪些位置。你不想一个一个翻。

查找和替换就是解决这个问题的。查找让你在整篇文档里快速定位某个词出现的位置。替换让你一次性把所有错误的词改成正确的。

4.6.1 查找

按 `Ctrl+F`（Mac 上是 `Cmd+F`），编辑器右上角会弹出一个搜索框。



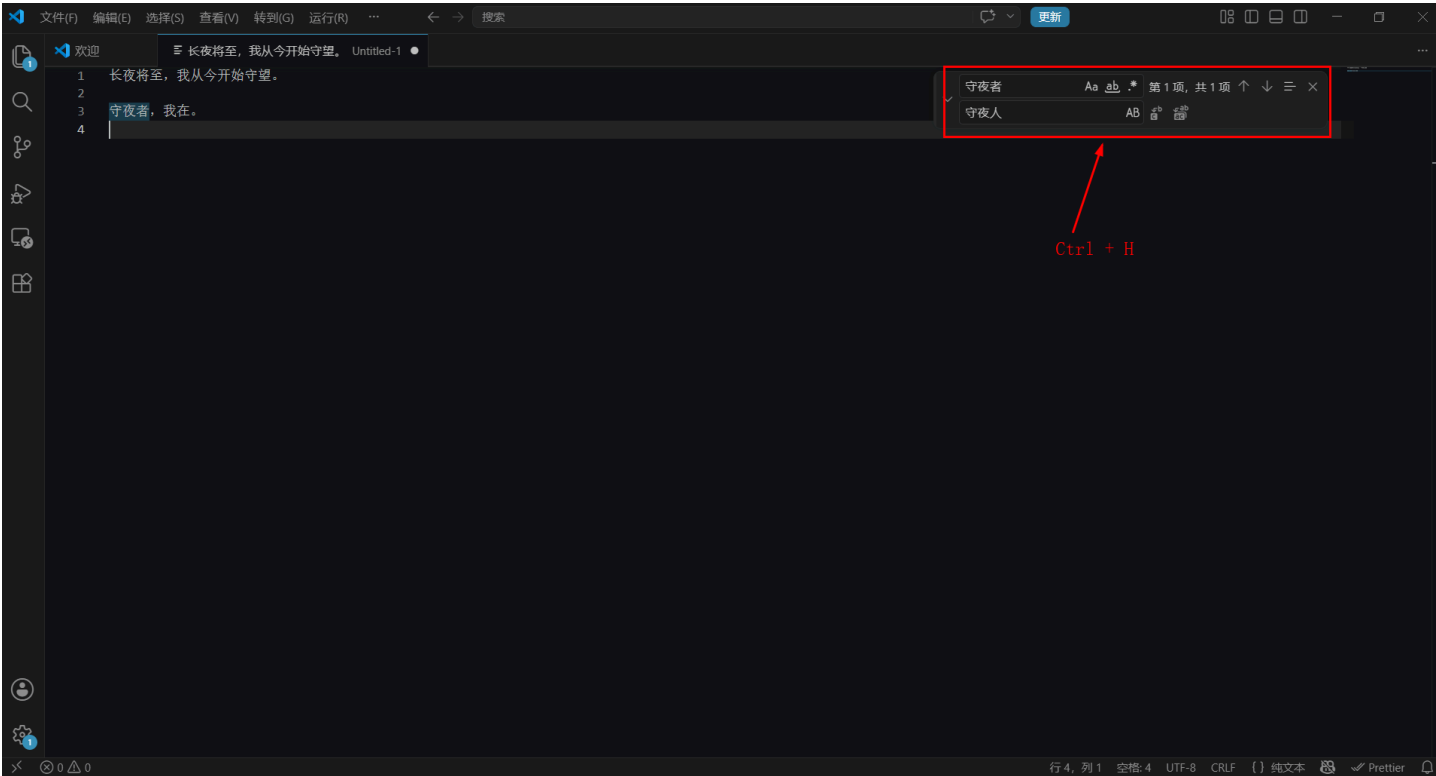
输入你要找的词，VS Code 会在当前文件里高亮所有匹配的位置。编辑区右侧的滚动条上会出现小色块，每一个色块对应一个匹配位置——你能看到它们在文档里的分布。

操作	快捷键	作用
跳到下一个匹配	Enter	光标跳到下一个匹配位置
跳到上一个匹配	Shift+Enter	光标跳到上一个匹配位置
关闭查找框	Esc	退出查找模式

Kate 按了 `Ctrl+F`，输入了 `Github`——文档里所有的“Github”都被高亮了。她说：“原来我写错了这么多次。不用一个一个翻，VS Code 全帮我找出来了。”

4.6.2 替换

按 `Ctrl+H`（Mac 上是 `Cmd+H`），编辑器右上角会弹出**替换框**——比查找框多一行。



第一行输入你要找的词（被替换的词），第二行输入你要换成什么词（替换成什么）。

操作	快捷键	作用
替换当前这一个	Enter（在第二行）	只替换光标当前所在的这个匹配
跳过这一个	Tab → Enter（在第二行）	不替换当前匹配，跳到下一个
替换所有	Ctrl+Alt+Enter	一次性替换文件里 所有 匹配

注意： Ctrl+Alt+Enter 会替换**整个文件**里所有匹配——包括你可能不想改的那些。如果你只改了部分匹配，建议先用 Enter 一个一个确认，或者先查找确认每个位置都该改，再用“替换所有”。

Kate 在替换框第一行输入了 Github，第二行输入了 GitHub，按了 Ctrl+Alt+Enter。文档里所有的“Github”瞬间变成了“GitHub”。她说：“六次写错，一次改完。我以前一个一个找、一个一个改，现在只需要按一次快捷键。”

4.6.3 查找和替换的选项

查找框和替换框右边有几个小按钮，用来控制搜索行为：

按钮	作用	什么时候用
区分大小写 (Aa)	开启后， github 不会匹配 GitHub	你想精确匹配大小写

按钮	作用	什么时候用
全字匹配 (Ab)	开启后，in 不会匹配 internal	你只想找独立的单词， 不想找到包含这个词的长词
正则表达式 (.*)	开启后， 可以用正则表达式搜索	你需要匹配某种模式——比如所有邮箱地址

前两个按钮对 Markdown 写作者最实用。比如你想把所有**单独出现的** `Git` 改成 `GitHub`，但不想把 `GitHub` 本身也匹配到——开启“全字匹配”，`Git` 就只会匹配独立的 `Git`，不会匹配到 `GitHub` 里的 `Git`。

Kate 开了“全字匹配”，输入了 `Git`，文档里只高亮了那些单独出现的 `Git`，`GitHub` 没有被高亮。她说：“原来‘全字匹配’就是告诉 VS Code——我找的是这个单词，不是这个字母组合。”

4.6.4 跨文件搜索

`Ctrl+F` 只能在当前打开的文件里查找。如果你想在**整个项目文件夹**里搜索某个词——比如你想找到所有提到“守夜者”的笔记文件，但你不知道这个词出现在哪几个文件里——按 `Ctrl+Shift+F`（Mac 上是 `Cmd+Shift+F`）。

左侧会打开搜索面板，输入你要找的词，VS Code 会列出项目里**所有包含这个词的文件**，以及每个匹配出现在文件的第几行。点一下搜索结果，自动打开那个文件并跳到对应位置。

你也可以在跨文件搜索里执行替换——点击搜索框右边的三个点（...），展开替换输入框，输入替换词，然后点“全部替换”。**这个操作不可撤销**——它会修改所有文件里的所有匹配。在执行跨文件全部替换之前，务必先用 `git commit` 保存当前状态，万一误操作可以用 `git checkout` 回退。

4.6.5 查找和 `Ctrl+D` 的区别

方式	适合场景	操作方式
<code>Ctrl+F</code> 查找	只是想看看某个词出现在哪些位置——不一定要改	搜索 → 浏览 → 关闭
<code>Ctrl+H</code> 替换	想把某个词全部或部分替换成另一个词	搜索 → 替换 → 确认
<code>Ctrl+D</code> 多光标	只想改某几个位置，而且你能一眼看到它们	选中 → <code>Ctrl+D</code> 逐个添加 → 手动修改

`Ctrl+D` 适合“我看到了，我想改这一处和那一处”。`Ctrl+H` 适合“我不知道它出现了多少次，我想全部或部分替换”。两者互补，不是替代。

4.6.6 养成肌肉记忆

快捷键	作用
Ctrl+F	在当前文件里查找
Ctrl+H	在当前文件里查找并替换
Ctrl+Shift+F	在整个项目里跨文件搜索

查找和替换是你写长文档时最常用的功能之一。下次你在笔记里发现一个写错的词，别手动一个一个改——先按 `Ctrl+H`，输入错误词和正确词，决定是逐个替换还是一次性全部替换。你的手指在键盘上完成这一切，不需要碰鼠标。

4.7 快捷键速查表

这一节是第四章的收尾。前面六节你逐个学完了最常用的快捷键——快速打开文件、命令面板、多光标、终端、整行移动、查找替换。这一节把它们汇总成一张速查表，方便你以后忘了随时回来翻。

按 `Ctrl+K Ctrl+S`（Mac 上是 `Cmd+K Cmd+S`）可以打开 VS Code 自带的快捷键编辑器，里面列出了所有快捷键，支持搜索和自定义修改。

4.7.1 核心快捷键速查

分类	快捷键	作用	出现章节
文件	Ctrl+P	快速打开文件	4.1
文件	Ctrl+Shift+Tab	在最近打开的文件之间切换	4.1.3
文件	Ctrl+W	关闭当前文件	3.3.3
命令	Ctrl+Shift+P	打开命令面板	4.2
命令	Ctrl+P 后输入 >	在快速打开文件时切换到命令面板	4.1.4
编辑	Ctrl+D	选中下一个相同词	4.3
编辑	Ctrl+K Ctrl+D	跳过当前匹配	4.3.2
编辑	Alt+Click	在任意位置添加光标	4.3.3
编辑	Alt+↑/↓	整行向上/向下移动	4.5

分类	快捷键	作用	出现章节
终端	<code>`Ctrl+``</code>	打开/关闭终端	4.4
终端	<code>`Ctrl+Shift+``</code>	新建终端	4.4.4
查找	<code>Ctrl+F</code>	当前文件查找	4.6.1
查找	<code>Ctrl+H</code>	当前文件查找并替换	4.6.2
查找	<code>Ctrl+Shift+F</code>	跨文件搜索	4.6.4
视图	<code>Ctrl+\</code>	分屏（左右）	3.3.1
视图	<code>Ctrl+K Ctrl+\</code>	分屏方向切换（左右↔上下）	3.3.1
视图	<code>Ctrl+1/2/3</code>	在分屏编辑器之间切换	3.3.2
视图	<code>Ctrl+Shift+Enter</code>	最大化/还原当前编辑器	3.3.3

4.7.2 顺手看一眼的辅助快捷键

这些快捷键不是“每天都用”的核心键，但在特定场景下能省很多事：

快捷键	作用	场景
<code>Ctrl+S</code>	保存文件	养成习惯：每写完一段就按一次
<code>Ctrl+Z</code>	撤销	写错了，回退一步
<code>Ctrl+Y</code>	重做	撤销多了，恢复一步
<code>Ctrl+B</code>	切换侧边栏显示/隐藏	侧边栏挡视线的时候关掉它
<code>Ctrl+J</code>	切换终端面板显示/隐藏	和 <code>Ctrl+``</code> 效果一样，但 <code>Ctrl+J</code> 会保留终端内容（不会因为关闭而清空）
<code>Ctrl+Shift+[</code>	折叠当前代码块/标题	长文档里折叠不需要看的部分
<code>Ctrl+Shift+]</code>	展开当前代码块/标题	折叠之后恢复展开
<code>Ctrl+K Z</code>	禅模式	全屏专注模式，侧边栏、状态栏、菜单全部隐藏。 按两次 <code>Esc</code> 退出
<code>Ctrl+K Ctrl+S</code>	打开快捷键编辑器	查看所有快捷键、搜索快捷键、自定义修改

4.7.3 打印版速查卡

把下面这张表截图保存，或打印出来贴在显示器旁边。前两周反复看，两周后这些快捷键会成为你的肌肉记忆。

频率	快捷键	作用
☆☆☆	Ctrl+P	快速打开文件
☆☆☆	Ctrl+Shift+P	命令面板
☆☆☆	Ctrl+D	多光标（选词）
☆☆☆	`Ctrl+``	打开终端
☆☆☆	Ctrl+H	查找并替换
☆☆	Ctrl+\	分屏
☆☆	Alt+↑/↓	整行移动
☆☆	Ctrl+Shift+F	跨文件搜索
☆	Ctrl+K Z	禅模式（全屏专注）

4.7.4 快捷键之间的“连招”

单个快捷键是招式，组合起来是连招。以下是几个典型的“连招”场景：

场景一：写笔记时发现一个词写错了

Ctrl+H → 输入错误词 → Tab → 输入正确词 → Ctrl+Alt+Enter 全部替换。

场景二：需要一边看参考文章一边写笔记

Ctrl+\ 分屏 → Ctrl+1 回到左边编辑器 → Ctrl+P 打开参考文章 → Ctrl+2 跳到右边编辑器 → Ctrl+P 打开自己的笔记。左右对照，互不干扰。

场景三：写完笔记，提交到 GitHub

Ctrl+`` 打开终端 → git add . && git commit -m "笔记更新" → git push → Ctrl+1` 回到编辑器。提交和写作在同一个窗口里完成，不需要切换应用。

Kate 试了一遍“场景二”的连招，说：“我以前做这件事要开两个窗口、来回 Alt+Tab、每次切换都重新找文件。现在六个快捷键，一气呵成。这不是快捷键，这是组合技。”

🎉 第四章，通关！

你现在已经可以：

- ✅ 用 `Ctrl+P` 在几秒内打开任何文件
- ✅ 用 `Ctrl+Shift+P` 命令面板执行任何命令
- ✅ 用 `Ctrl+D` 同时修改多个相同词
- ✅ 用 `Ctrl+`` 随时打开终端
- ✅ 用 `Ctrl+H` 查找并替换文档里的任何内容
- ✅ 用 `Alt+↑/↓` 快速移动整行文字

这些快捷键不是用来“炫技”的——它们是用来让你把更多时间花在**写内容**上，而不是花在**找文件、改错字、切窗口**上。把上面那张“打印版速查卡”贴在显示器旁边。前两周反复看，两周后它们会成为你的肌肉记忆。

Kate 把她打印出来的速查卡贴在了显示器旁边。她说：“从今天开始，我每用一个快捷键就在旁边打个勾。两周后，我要让这张纸上的所有键都变成我的手指本能。”

“等你全部打勾了，你就不再是‘用 VS Code 写 Markdown 的人’——你是‘手指比大脑更快的人’。那不是炫技，那是自由。”

🎉 《玩转 VS Code》，全部通关！

你从一台刚装好的 VS Code 开始，学会了：

- ✅ 第一章：在三个系统上安装 VS Code，配置 `code` 命令，调整个性化设置
- ✅ 第二章：认识活动栏、编辑区、终端、状态栏、命令面板——不再对着屏幕一脸懵
- ✅ 第三章：快速打开文件、多光标编辑、分屏写作、自动格式化、自动保存——用键盘速度碾压鼠标流
- ✅ 第四章：`Ctrl+P`、`Ctrl+Shift+P`、`Ctrl+D`、`Ctrl+``、`Alt+↑/↓`、`Ctrl+H`——六个核心快捷键，加上一张可打印的速查表

这不是一本“教你装 VS Code”的教程。这是一本**让你的手指比大脑更快的驾驶手册**。

Kate 在她的笔记本封底写了一行字：“从‘这是什么东西’到‘手指比大脑快’，用了四章。”

守夜人，VS Code 毕业了。

《守夜者之书》第二册：《玩转 VS Code》—— 终。